WEB PILOT

Junaid Ul Hassan

Ubaid Ur Rehman

Anique Khadim

Supervise By

Mr. Muhammad Waqar

Submitted for the partial fulfillment of BS Software Engineering degree to the Faculty of Engineering & Computing

NATIONAL UNIVERSITY OF MODERN LANGUAGES ISLAMABAD

March, 2025

ABSTRACT

People often struggle to find useful information because they have to go through long texts, modern websites, youtube videos, and PDFs. Managing and searching through large amounts of content can be time-consuming and frustrating. Users may lose interest, experience poor service, or even leave the website. To solve this issue, they need a smart and easy-to-use tool that helps them quickly access and retrieve important information, saving both time and effort.

Leo Brave AI and HuggingChat were two alternatives, but they had limitations. Leo Brave only worked within a browser and had a limited range. Hugging chat was not specialized; it could only process some youtube videos and did not support the latest information, PDFs, or all youtube videos. These restrictions showed the need for a more advanced solution, especially for users looking for information from specific websites. Due to these limitations, a better solution was needed to provide users with a more advanced, flexible, and efficient way to access information. To fill this gap, Web Pilot was introduced as a powerful alternative. It designed as a chrome extension with an intelligent chatbot capable of interacting with various online content sources, including youtube videos, PDFs (both online and offline), and web pages. Unlike previous solutions, Web Pilot continuously learned from user interactions, improving its responses over time. It could extract and understand text from different formats, making browsing more interactive and user-friendly. By allowing users to chat with the tool using simple text commands, Web Pilot ensured quick and accurate information retrieval, making it a reliable and efficient assistant for online research and content navigation.

Web Pilot was tested carefully to ensure it worked correctly. The black-box testing method was used to check its functionality without looking at the internal code. Each part of the application went through unit testing to confirm that it worked properly on its own. After that, integration testing was done to check if all parts worked well together. A set of test cases was created to test how Web Pilot handled different types of content, such as youtube videos, PDFs, and web pages. The results showed that Web Pilot worked smoothly, provided accurate responses, and met all the necessary standards for a better and more interactive browsing experience.

CERTIFICATE

Dated: _____

Final Approval

It is certified that project report titled **Web Pilot** submitted by **Junaid Ul Hassan**, **Ubaid Ur Rehman** and **Anique Khadim** for the partial fulfillment of the requirement of "**Bachelor's Degree in Software Engineering"** is approved.

COMMITTEE

Dr. Muhammad Noman Malik

DEAN FEC Signature: _____

Dr. Sumaira Nazir

HoD Software Engineering Signature: _____

Dr. Naveed Ahmad

Head Project Committee Signature: _____

Mr. Muhammad Waqar

Supervisor Signature: _____

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group Members	Signature
1. Junaid Ul Hassan	_____
2. Ubaid Ur Rehman	_____
3. Anique Khadim	_____

PLAIGRISM CERTIFICATE

This is to certify that the project entitled "**Web Pilot**", which is being submitted here with for the award of the "Degree of Bachelor" in "Software Engineering". This is the result of the original work by **Junaid Ul Hassan**, **Ubaid Ur Rehman** and **Anique Khadim** under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of the award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report

Processed on: 12-Jan-2025 13:12 PKT
ID: 2561755368
Word Count: 9591

Similarity Index
7%
Similarity by Source
Internet Source: 6%
Publications: 2%
Student Papers: N/A

Date: 12/01/2025

_____

Mr. Muhammad Waqar

TURNITIN ORIGINALITY REPORT

Turnitin Originality Report

Processed on: 12-Jan-2025 13:12 PKT
ID: 2561755368
Word Count: 9591
Submitted: 4

Web Pilot By Naveed Ahmad

Similarity Index	Similarity by Source
7%	Internet Sources: 6% Publications: 2% Student Papers: N/A

1% match (Internet from 13-Dec-2024)
https://www.coursehero.com/file/155829069/Web-Inspector-Muhammad-Waqar/

1% match (Internet from 28-Aug-2021)
https://www.coursehero.com/file/41551916/Design-and-Fabrication-of-rectenna-originalpdf/

< 1% match (Internet from 18-Mar-2023)
https://WWW.coursehero.com/file/61248358/medical-store-softwaredoc/

< 1% match (Internet from 25-Sep-2023)
https://www.coursehero.com/file/121456134/Group-19-BSSE-24-Expense-Tracker-Final-Proposal-Documentdoc/

< 1% match (Internet from 02-Dec-2024)
https://WWW.coursehero.com/file/155829700/FINAL-REPORT-VERSION/

< 1% match (Internet from 20-Feb-2022)
https://www.coursehero.com/file/123810404/HU-staff-finance-management-systempdf/

< 1% match (Internet from 12-Aug-2024)
https://www.coursehero.com/file/p36q351d/There-are-various-categories-of-the-IoT-including-the-industrial-Internet-of/

< 1% match (Internet from 10-Dec-2023)
https://core.ac.uk/download/11039038.pdf

< 1% match (Internet from 03-Dec-2024)
https://www.bomberbot.com/javascript/how-to-create-a-chrome-extension-a-comprehensive-guide/

< 1% match (Internet from 23-Feb-2024)
https://numl.edu.pk/offices/plagiarism/1697782291SOP-FYP-SE%20Dept-Oct2023.pdf

---

< 1% match (Internet from 08-Oct-2022)
https://numl.edu.pk/offices/plagiarism/1609084680SOP%20FYP%20EE%20NUML%20revised%20Feb%202020.pdf

---

< 1% match (Internet from 02-Aug-2023)
https://ebin.pub/azure-modern-data-architecture-a-guide-to-design-and-implement-a-modern-data-solutions.html

---

< 1% match (publications)
Shravan Pargaonkar, "A Guide to Software Quality Engineering", CRC Press, 2024

< 1% match (Internet from 11-Apr-2009)
http://www.click.up.ac.za/Resources/Student_resources/Discussions.pdf

---

< 1% match (Internet from 17-May-2024)
https://www.hospitalitynet.org/opinion/4117195.html

< 1% match (Internet from 26-Feb-2023)
https://ieeexplore.ieee.org/document/9788276/

< 1% match (Internet from 24-Mar-2024)
https://www.architecturemaker.com/what-is-internet-architecture/

TABLE OF CONTENTS

Chapter **Page**

CHAPTER 1: INTRODUCTION....14

1.1 Introduction....15

1.2 Problem Description....15

1.3 Project Objective....15

1.4 Developed System....15

1.4.1 Features....16

1.4.2 Expected Result....16

1.5 Modules....16

1.5.1 Data Scraping....16

1.5.2 Chatbots Integration....16

1.5.3 Flow chart of working System....17

1.6 Project Scope....18

1.7 Feasibility Study....18

1.7.1 Risk involved....18

1.7.2 Nature of Project....20

1.8 Tools & Technology....20

1.8.1 Software....20

1.8.2 Hardware....20

1.9 Methodology....20

1.10 Milestones....22

1.11 Summary....22

CHAPTER 2: BACKGROUND AND EXISTING WORK....23

2.1 Introduction....24

2.2 Existing Systems....24

2.3 Leo....24

2.3.1 Features....24

2.3.2 Limitations....24
2.4 Hugging Chat....25
2.4.1 Features....25
2.4.2 Limitations....25
2.5 Comparison between Existing and Proposed System....25
2.6 Resource Requirements....26
2.7 Tools & Technology....26\
2.7.1 Software....26
2.7.2 Hardware....26
2.6 Summary....26
CHAPTER 3: REQUIREMENTS SPECIFICATION....27
3.1 Introduction....28
3.2 Functional Requirements....28
3.3 Non-Functional Requirement:....34
3.3.1 Usability (Operability)....34
3.3.2 Reliability (Fault Tolerance)....34
3.3.3 Efficiency (Time Behavior)....34
3.3.4 Maintainability (Changeability, Stability)....34
3.4 Use Case Modelling....34
3.5 Use Cases Description....36
3.5.1 Use case: Web Page....36
3.5.2 Use case: On Plugin....36
3.5.3 Use case: Chatbot Open....37
3.5.4 Use case: Enter Query....37
3.5.5 Use case: Handle Invalid Input....38
3.5.6 Use case: Response Generate....38
3.5.7 Use case: Show Answer....39
3.5.8 Use case: Off Chatbot....40
3.5.9 Use case: Terminate Data....40

3.5.10 Use case: Text Scrape....41
3.5.11 Use case: Preprocess Data....41
3.5.12 Use case: Convert Data into Vector....42
3.5.13 Use case: Store in Vector DB....42
3.5.14 Use case: Model Train....42
3.5.15 Use case: Save History....43
3.6. Summary....43
CHAPTER 4: SYSTEM MODELING....44
4.1 Introduction....45
4.2 (4+1) View Model....45
4.2.1 Logical View....46
4.2.1.1 UML Class Diagram....46
4.2.2 Process View....47
4.2.2.1 UML Activity Diagram....47
4.2.3 Sequence Diagram....48
4.2.3.1 User Sequence Diagram....49
4.2.3.2 Response Generate Sequence Diagram....50
4.2.4 Development View....51
4.2.4.1 UML Component Diagram....51
4.2.5 Physical View....52
4.2.5.1 UML Deployment Diagram....52
4.2.6 System Prototype....53
4.2.6.1 Initial Page....53
4.2.6.2 Home Page....54
4.2.6.3 Chat Page....55
4.2.6.3 Model Information....56
4.7 Summary....56
CHAPTER 5: IMPLEMENTATION....57
5.1 Introduction....58

5.2 Modules of the System....58
5.2.2 Browser Extension....58
5.2.1 Conversation Panel....59
5.2.3 Content Extraction....59
5.2.4 Data Preprocessing and Parsing....59
5.2.5 Retrieval-Augmented Generation (RAG)....60
5.2.5.1 Create Documents....60
5.2.5.2 Split Documents....60
5.2.5.3 Text Embeddings....60
5.2.5.4 Vector Database....61
5.2.5.5 Chat Memory Management....61
5.2.6 Large Language Model Integration....61
5.2.8 Error Handling....62
5.3 Frameworks....64
5.3.1 Chrome Extension Development....64
5.3.1.1 Load Unpacked Extension....64
5.3.1.2 Structure an Extension Project....66
5.3.2 HTML/CSS/JS....66
5.3.3 Flask (Python Backend-Framework)....66
5.3.5 Langchain....67
5.3.6 Docker....67
5.3.6.1 Base Image....68
5.3.6.2 System Dependencies....68
5.3.6.3 Working Directory....68
5.3.6.4 Copying Application Files....68
5.3.6.5 Python Dependencies....68
5.3.6.6 Environment Variables....69
5.3.6.7 Port Exposure....69
5.3.6.8 Command to Run the Application....69
5.3.7 AWS Auto scaling Group (ASG)....69

5.4. Summary....69
CHAPTER 6: TESTING, ANALYSIS AND VALIDATION....70
6.1 Introduction....71
6.2 Test Methodology....71
6.3 Test Bed....71
6.4 Test Cases....71
6.4.1 Plugin Usage on Any Website....72
6.4.2 Plugin Usage on Any PDF File....73
6.4.3 Plugin Usage on Any YouTube Video....74
6.5 Summary....74
CHAPTER 7: CONCLUSION AND FUTURE WORK....75
7.1 Introduction....76
7.2 System Overview....76
7.3 Milestone Achieved....77
7.3.1 Develop Browser extension....77
7.3.2 LLM Based Chatbot....77
7.3.3 Dynamically Train on any text data....77
7.3.3 Chat with any Large Contents....77
7.4 Limitations....77
7.5 Future Work....77
7.6 Summary....78
APPENDICES....79
Appendix - I: Glossary....79
REFERENCES....81

LISTS OF FIGURES

Figure	Caption	Page
1.1:	Flow chart	18
1.2:	Process Model	22
3.1:	Use Case Diagram	36
4.1:	4+1 View Model	47
4.2:	Class Diagram	48
4.3:	UML Activity diagram	49
4.4:	User Sequence Diagram	50
4.5:	Response Generation Sequence Diagram	51
4.6:	Component Diagram	52
4.7:	Deployment Diagram	53
4.8:	Web Pilot Initial Page	54
4.9:	Web Pilot Home Page	55
4.10:	Web Pilot Chat Page	56
4.11:	Model information	57
5.1:	Chrome Extension	59
5.2:	Data Preprocessing and Parsing	60
5.3:	Chat memory	62
5.4:	LLM Integration	63
5.5:	Chat with PDF	64
5.6:	Chat with YouTube videos	64
5.7:	Load Upacked Extension	65
5.8:	Manifest code	66
5.9:	Extension file structure	67
5.10:	Langchain Libraries	68

LISTS OF TABLES

Table	Caption	Page
1.1:	Risks Involved	20
1.2 :	Milestones	23
2.1:	Comparative Analysis among Existing Systems	26
3.1:	Chrome Extension Integration Functional Requirement	29
3.2:	AI Chatbot Interaction Functional Requirement	29
3.3:	Web URL Retrieval Functional Requirement	30
3.4:	Website Content Scraping Functional Requirement	30
3.5:	YouTube Video Content Scraping Functional Requirement	30
3.6:	PDF Content Scraping Functional Requirement	31
3.7:	LLM Model Training Functional Requirement	31
3.8:	User Query Input Functional Requirement	32
3.9:	LLM-Generated Answers Functional Requirement	32
3.10:	Chatbot Interface Functional Requirement	32
3.11:	Embedding Algorithm Updates Functional Requirement	33
3.12:	Error Handling and Feedback Functional Requirement	33
3.13:	User Query Logs Functional Requirement	33
3.14:	Help and Support Documentation Functional Requirement	34
3.15:	System Notifications Functional Requirement	34
3.16:	Web Page	34
3.17:	On Plugin	37
3.18:	Chatbot Open	37
3.19:	Enter user query	38
3.20:	Handle Invalid User input	38
3.21:	Response Generate	39
3.22:	Show LLM Answer	39
3.23:	Swith off chatbot	40
3.24:	Terminate data	41
3.25:	Text Scrapping	41

3.26: Preprocess Data.....42
3.27: Convert data into vectors.....42
3.28: Store in Vector DB.....43
3.29: Model Training.....43
3.30: Save History.....44
6.1: Test case Usage on Any Website.....73
6.2: Test Case chat with pdf file.....74
6.3: Test case Usage on Any YouTube Video.....75

CHAPTER 1

INTRODUCTION

1.1 Introduction

This undertaking introduces a progressive Chrome extension offering a bendy AI chatbot designed for seamless integration with any internet site, PDF files, and YouTube films. The chatbot is prepared with superior abilities, including textual content and large content extraction, in addition to an revolutionary schooling technique referred to as Retrieval-Augmented Generation (RAG) thru huge language fashions (LLM) and Generative AI. This permits the chatbot to quickly examine from man or woman interactions, ensuring correct and context-sensitive responses. Users can have interaction with the chatbot through conventional text input, growing a herbal and interactive surfing revel in. The chatbot's sturdy functionality lets in clients to inquire approximately net site content material and photograph data, providing particular and knowledgeable solutions. This extension hobbies to redefine how customers interact with online systems, presenting a custom designed and intelligent assistant across a giant variety of internet site.

1.2 Problem Description

Browsing a internet site may be tough, specifically with plenty of stuff to go through like words and photographs. People often war to discover what they want, losing time and energy. The way a website is installation can make it even more difficult to discover specific information. This is a massive hassle, mainly whilst people are looking for data on elaborate subjects or don't have a lot time. So, many site visitors emerge as leaving the website feeling disappointed and sad and that they won't come returned. We trust there's a need for a easier and friendlier way to browse and our solution goals to make it less complicated for each person to determined what they may be seeking out, making the whole enjoy better and extra gratifying.

1.3 Project Objective

Our project main goals is to develop a chrome extension which can be open at any webpage, youtube videos or pdf files and it will efficiently reading both all large contents in seconds

1.4 Developed System

We are developed a AI chatbot Chrome extension using advance LLMs and Generative AI.

It can extracts text from any open web-content, youtube video or pdfs, then trains the chatbot with that data. Users can ask the chatbot questions about the websites and it responds based on the content it learned.

1.4.1 Features

- Chrome Extension
- AI Based Chat Bot
- Adaptive Integration
- Extract text data
- Answering Questions

1.4.2 Expected Result

- Give Accurate Response
- User Friendly
- Summarization
- Efficient text extraction
- Answering Questions according to web content

1.5 Modules

1.5.1 Data Scraping

This workflow diagram provides a visual representation of the sequential steps involved in the data scraping module, from initiation to completion.

1.5.2 Chatbots Integration

The workflow start with user input and the chatbot processes it through text fetching from large contents and then generating appropriate responses. If the user request any types of details, the chatbot provides specific information, conclude with the end of user interaction. The diagram below shows the seamless flow from input to response, emphasize the handling of text-related queries.

1.5.3 Flow chart of working System

The Figure 1.1 flowchart outlines the operational workflow of the WebPilot application, detailing the user interaction and subsequent processing to generate output. It visually represents the steps a user takes, from initial input to the final result, illustrating the application's internal mechanisms and the transformation of user requests into actionable responses. Essentially, it maps the journey from user query to WebPilot's delivered output.

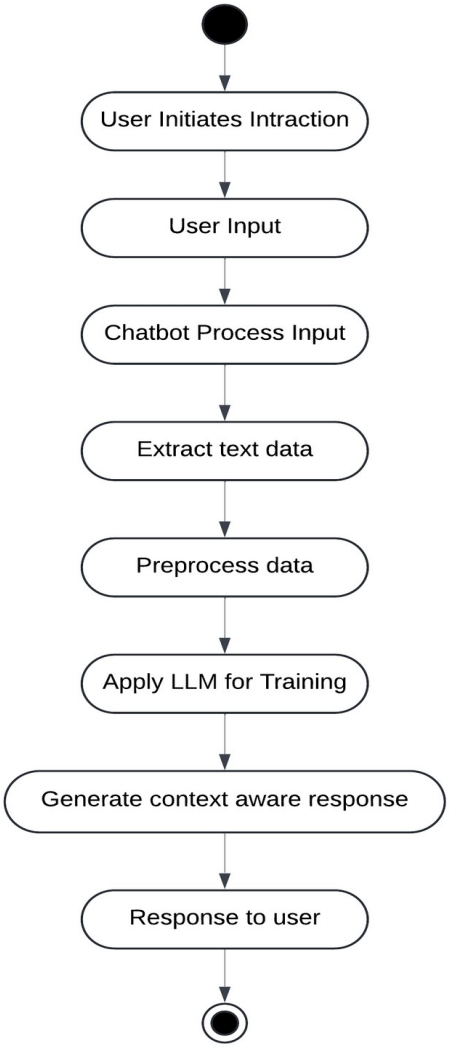


Figure 1.1: Flow chart

1.6 Project Scope

Our project aims to develop an advanced Chrome extension integrated with an AI-powered chatbot that enhances users' ability to access and interact with web content. This extension will be designed to extract and analyze text from a wide range of online sources, including websites, YouTube videos, and PDF files, providing users with a seamless and efficient browsing experience.

The extension will enable users to retrieve information quickly by processing text from dynamic and static web pages, transcribing spoken content from YouTube videos, and extracting data from both online and offline PDF documents. The AI chatbot will act as an intelligent assistant, allowing users to ask questions and receive accurate, context-aware responses.

Proposed system will satisfy the following ranges:

- Universal Applicability
- Efficient Data Extraction
- Dynamic Learning Capability
- User-Friendly Interaction
- Information Precision

1.7 Feasibility Study

We will try our possible best to meet the project schedules, but there are some risk factors involved that are mentioned below:

1.7.1 Risk involved

Our application may face several challenges while extracting and processing data. One major limitation is the inability to retrieve information from restricted websites—those that explicitly prohibit data extraction. These websites may have strict security measures, such as CAPTCHA verification, login requirements, or anti-scraping technologies, which prevent automated tools from accessing their content.

Another potential challenge is internet connectivity issues. If the user experiences a weak or unstable internet connection, the system may struggle to fetch real-time data, leading to

delays or incomplete responses. This could impact the overall user experience and limit the chatbot's ability to provide timely and relevant information. The system may occasionally provide inaccurate or incomplete responses due to incorrect data sources, text misinterpretation, or limitations in natural language understanding. While Web Pilot is designed to improve through continuous learning, errors can still occur. These inaccuracies arise from various factors, including flawed input data, contextual misunderstandings, or inherent complexities in language processing. The system's ability to interpret and generate responses depends on the quality and reliability of the data it processes. Addressing these challenges requires ongoing refinements, optimizations, and advancements in AI capabilities. Continuous updates and improvements aim to enhance accuracy and reliability over time. However, achieving perfect precision remains a challenge due to the dynamic nature of language and diverse user queries. Ensuring better performance involves refining algorithms, improving data validation, and incorporating user feedback.

Table 1.1: Risks Involved

No.	Key Risks	Hypothetical Statement	Resolution
1	Data Privacy Concerns	The extraction process might inadvertently capture sensitive or personal information, raising privacy concerns.	Implement robust data anonymization techniques and ensure strict adherence to privacy regulations to safeguard user data.
2	Misinterpretation of Content	Dynamic web content could lead to inaccuracies in the chatbot's responses, potentially causing misinformation and affecting user trust.	Regularly refine and enhance the chatbot's understanding through continuous learning mechanisms to minimize inaccuracies and build user confidence.

1.7.2 Nature of Project

This Chrome extension will be applicable on all type of websites and chatbot working on all type of text data.

1.9 Methodology

For this project, we have chosen the Iterative Waterfall methodology, which provides a structured yet flexible approach to software development. This model follows a sequence of well-defined phases, including planning, design, implementation, testing, and deployment, ensuring that each stage is completed before moving on to the next. However, unlike the traditional Waterfall model, our approach incorporates iteration and refinement, allowing us to revisit and improve previous phases whenever necessary. This iterative aspect ensures that the system can evolve based on new insights, user feedback, or changing requirements without disrupting the overall development process.

The planning phase focuses on understanding project requirements, setting objectives, and defining the scope of work. The design phase involves creating detailed architecture, UI/UX layouts, and technical specifications, ensuring that all components align with the project's goals. During implementation, the system is developed in a modular fashion, allowing for efficient integration of features and functionalities. The testing phase ensures that each component functions correctly through rigorous unit testing, integration testing, and system testing. Finally, in the deployment phase, the completed and verified system is released for use, with continuous monitoring and maintenance to address any post-deployment issues.

One of the key advantages of using this methodology is its balance between structure and adaptability. By allowing iterations, we can refine functionalities, address any gaps, and improve system performance without waiting until the final stage. This structured yet flexible approach helps maintain clear development steps, reduces errors, and enhances system reliability. Additionally, early detection and resolution of issues prevent major setbacks, ensuring the final product is well-refined, fully functional, and of high quality. By following this methodology, we ensure that our project meets its objectives efficiently

while maintaining transparency, organization, and continuous improvement throughout the development process.

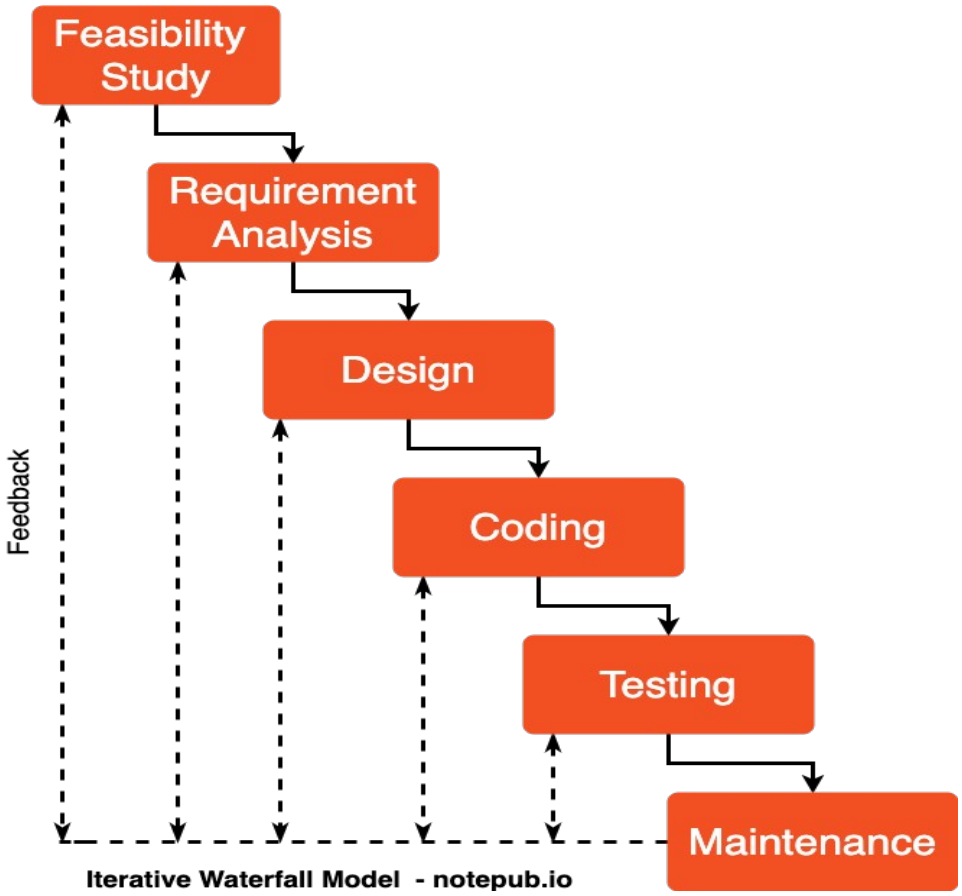


Figure 1.2: Process Model

1.10 Milestones

Table 1.2 outlines the project milestones for the year 2023-24, detailing seven tasks and their respective timelines across several months. The table uses a visual representation to show the duration of each task, with "Test and training of AI model" spanning the longest period from February to May.

Table 1.2: Milestones

	Tasks	Year 2023-24								
S/No	Title	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun	Sep
1	Project Planning and Research									
2	Design of Chrome Extension									
3	Development of Chatbot									
4	Test and training of AI model									
5	System Integration									
6	System Testing									
7	System Documentation									

1.11 Summary

This chapter provides an AI chatbot that integrates with websites, PDFs, and YouTube videos. It uses advanced AI, including RAG and LLM, to understand and respond to user queries with contextually relevant information. Users can interact via text, receiving intelligent answers about website content and images, enhancing their browsing experience.

CHAPTER 2

BACKGROUND AND EXISTING WORK

2.1 Introduction

This project introduces a revolutionary Chrome extension featuring a versatile AI chatbot designed for seamless integration with any website. The chatbot is equipped with advanced capabilities, including text and image extraction, coupled with an innovative training approach called Lifelong Learning through (LLM). This enables the chatbot to dynamically learn from user interactions, ensuring accurate and context-sensitive responses. Users can engage with the chatbot through traditional text input or voice messages, facilitating a natural and interactive browsing experience. The chatbot robust functionality allows users to inquire about website content and image data, receiving precise and informed answers. This extension aims to redefine the way users interact with online platforms, offering a personalized and intelligent assistant across diverse websites.

2.2 Existing Systems

Currently, there are some question answering systems available, but they have limitations. They are often platform-dependent, which means they only work on specific Web Browsers. Additionally, they typically rely solely on text data and content, neglecting the wealth of information contained in youtube videos and pdfs.

2.3 Leo

Leo is Brave AI chatbot which extracts text from web pages and utilizes this data to train its models on the fly, enabling it to provide insightful answers to user queries about the content within those pages.

2.3.1 Features

- Summarizing any large web-content
- Answering questions
- Generate Texts

2.3.2 Limitations

- Advance features are not free.
- Only available on Brave Browser

- Cannot describe Images

2.4 Hugging Chat

HuggingChat is a state-of-the-art language model that can answer and generate questions about various topics by leveraging real-time data from websites. When information isn't readily available, HuggingChat can search the web to find the most up-to-date and relevant data to provide accurate responses.

2.4.1 Features

- Web Search
- Text Translation
- Answering Questions
- Generate Text

2.4.2 Limitations

- Not working on specific website
- Limited Domain knowledge
- Scrape text from random websites
- No latest information

2.5 Comparison between Existing and Developed System

Table 2.1 presents a comparative analysis of three systems.

Table 2.1: Comparative Analysis among Existing Systems

Sr.	Criteria	Leo	Hugging Chat	Developed System
1.	Cost (Expensive)	Yes	No	No
2.	Read Youtube Videos	No	No	Yes
3.	Fetch data from specific Source	Yes	No	Yes
4.	Latest Information	Yes	No	Yes
5.	Chrome Extension	No	No	Yes
6.	Read PDFs	No	No	Yes

2.6 Resource Requirements

Following are the resource requirements for the system.

2.7 Tools & Technology

Following tools and technology will be required in development of the Project

2.7.1 Software

- Jupyter notebook
- Pycharm IDE
- TensorFlow/PyTorch
- Visual Studio Code
- Docker
- AWS
- Kubernet

2.7.2 Hardware

- PC/ Laptop core i7 6th Gen and Above for better performance in programming
- GPU used for AI model trainings

2.6 Summary

In this chapter, we have examined key aspects of the application's structure and functionality. Additionally, we have reviewed the shortcomings of existing systems. Finally, we conclude with a comparative analysis of our system in relation to the existing systems.

CHAPTER 3

REQUIREMENTS SPECIFICATION

3.1 Introduction

This section is dedicated to specifying the system requirements. It covers both functional and non-functional requirements in detail, using use case diagram to provide a comprehensive analysis. This section includes a thorough documentation of the requirements to ensure clarity and understanding.

3.2 Functional Requirements

These are the following functional requirements of the system**.**

3.2.1 Chrome Extension Integration

Table 3.1 shows the Chrome Extension Integration functional requirement.

Table 3.1: Chrome Extension Integration Functional Requirement

Identifier	**FR-01**
Title	Chrome Extension Integration
Description	The system will be implemented as a Chrome extension that integrates seamlessly with any web content.
Priority	High

3.2.2 AI Chatbot Interaction

Table 3.2 shows the AI Chatbot Interaction functional requirement.

Table 3.2: AI Chatbot Interaction Functional Requirement

Identifier	**FR-02**
Title	AI Chatbot Interaction
Description	The extension will include an AI chatbot capable of interacting with users via text inputs.
Priority	High

3.2.3 Active Tab Detection

Table 3.3 shows the Active Tab Detection functional requirement.

Table 3.3: Active Tab Detection Functional Requirement

Identifier	**FR-03**
Title	Active Tab Detection
Description	The extension will be able to detect the URL of the currently active tab in the user's browser.
Priority	High

3.2.4 Web URL Retrieval

Table 3.4 shows the Web URL Retrieval functional requirement.

Table 3.4: Web URL Retrieval Functional Requirement

Identifier	**FR-04**
Title	Web URL Retrieval
Description	The system will fetch the web URL link of the currently open tab from the Chrome extension.
Priority	High

3.2.5 Website Content Scraping

Table 3.5 shows the Website Content Scraping functional requirement.

Table 3.5: Website Content Scraping Functional Requirement

Identifier	**FR-05**

Title	Website Content Scraping
Description	The system will scrape all text content from the currently open website.
Priority	High

3.2.6 YouTube Video Content Scraping

Table 3.6 shows the YouTube Video Content Scraping functional requirement.

Table 3.6: YouTube Video Content Scraping Functional Requirement

Identifier	**FR-06**
Title	YouTube Video Content Scraping
Description	The system will scrape all text transcript and data from the currently open YouTube video.
Priority	High

3.2.7 PDF Content Scraping

Table 3.7 shows the PDF Content Scraping functional requirement.

Table 3.7: PDF Content Scraping Functional Requirement

Identifier	**FR-07**
Title	PDF Content Scraping
Description	The system will scrape all text content from the currently open PDF file in the browser.
Priority	High

3.2.8 LLM Model Training

Table 3.8 shows the LLM Model Training functional requirement.

Table 3.8: LLM Model Training Functional Requirement

Identifier	**FR-08**
Title	LLM Model Training
Description	The system will train an LLM model on all scraped text content from the currently open website.
Priority	High

3.2.9 User Query Input

Table 3.9 shows the User Query Input functional requirement.

Table 3.9: User Query Input Functional Requirement

Identifier	**FR-09**
Title	User Query Input
Description	Users will be able to input queries about the currently open tab into the chatbot.
Priority	High

3.2.10 LLM-Generated Answers

Table 3.10 shows the LLM-Generated Answers functional requirement.

Table 3.10: LLM-Generated Answers Functional Requirement

Identifier	**FR-10**
Title	LLM-Generated Answers
Description	The system will generate an answer based on the context provided by the text and

	the user query.
Priority	High

3.2.11 Chatbot Interface

Table 3.11 shows the Chatbot Interface functional requirement.

Table 3.11: Chatbot Interface Functional Requirement

Identifier	**FR-11**
Title	Chatbot Interface
Description	Users will be able to view the LLM-generated answer within the chatbot interface.
Priority	High

3.2.12 Embedding Algorithm Updates

Table 3.12 shows the Embedding Algorithm Updates functional requirement.

Table 3.12: Embedding Algorithm Updates Functional Requirement

Identifier	**FR-12**
Title	Embedding Algorithm Updates
Description	The system will support updates and improvements to the embedding algorithm to ensure high relevance in query matching.
Priority	Medium

3.2.13 Error Handling and Feedback

Table 3.13 shows the Error Handling and Feedback functional requirement.

Table 3.13: Error Handling and Feedback Functional Requirement

Identifier	**FR-13**
Title	Error Handling and Feedback
Description	The system will handle errors and provide appropriate user feedback if the text scraping or vector retrieval process fails.

Priority	High

3.2.14 User Query Logs

Table 3.14 shows the User Query Logs functional requirement.

Table 3.14: User Query Logs Functional Requirement

Identifier	**FR-14**
Title	User Query Logs
Description	Users will be able to view a log of their past queries and interactions with the chatbot.
Priority	Medium

3.2.15 Help and Support Documentation

Table 3.15 shows the Help and Support Documentation functional requirement.

Table 3.15: Help and Support Documentation Functional Requirement

Identifier	**FR-15**
Title	Help and Support Documentation
Description	Users will be able to access help and support documentation within the extension.
Priority	Medium

3.2.16 System Notifications

Table 3.16 shows the System Notifications functional requirement.

Table 3.16: System Notifications Functional Requirement

Identifier	**FR-16**
Title	System Notifications
Description	Users will be able to see notifications for system updates and new features within the extension.

Priority	Medium

3.3 Non-Functional Requirement:

The project's success depends on focusing on important aspects, using non-functional requirements like sub-characteristic to ensure optimal performance and user satisfaction:

3.3.1 Usability (Operability)

Ensuring the system's ease of operation is essential for providing users with an intuitive interface, enabling seamless interaction with the chatbot across various websites.

3.3.2 Reliability (Fault Tolerance)

Robust fault tolerance mechanisms are vital to maintaining system stability and availability, ensuring uninterrupted access to the chatbot and building user trust.

3.3.3 Efficiency (Time Behavior)

Optimizing time behavior guarantees swift response times, meeting performance expectations and enhancing user satisfaction during browsing sessions.

3.3.4 Maintainability (Changeability, Stability)

Prioritizing changeability and stability facilitates easy updates and modifications to the system, ensuring adaptability and consistent performance over time, contributing to long-term success.

3.4 Use Case Modelling

Use case modeling, a foundational practice in software development, provides a clear and comprehensive visualization of how a system interacts with its intended users and other stakeholders. This methodology employs use case diagrams and fully dressed use cases to meticulously document the system's functionality and the actors involved. A use case diagram, as depicted in Figure 3.1, offers a high-level overview, illustrating the system's boundaries, the various actors (users, external systems, etc.), and the primary interactions or "use cases" that define the system's behavior. Each actor's role and their interaction with the system are visually represented, outlining the scope of the system's capabilities.

Complementing the diagram, fully dressed use cases provide detailed, textual descriptions of each interaction. These descriptions delve into the step-by-step processes involved in each use case, including the normal flow of events, alternative flows, and exception handling.

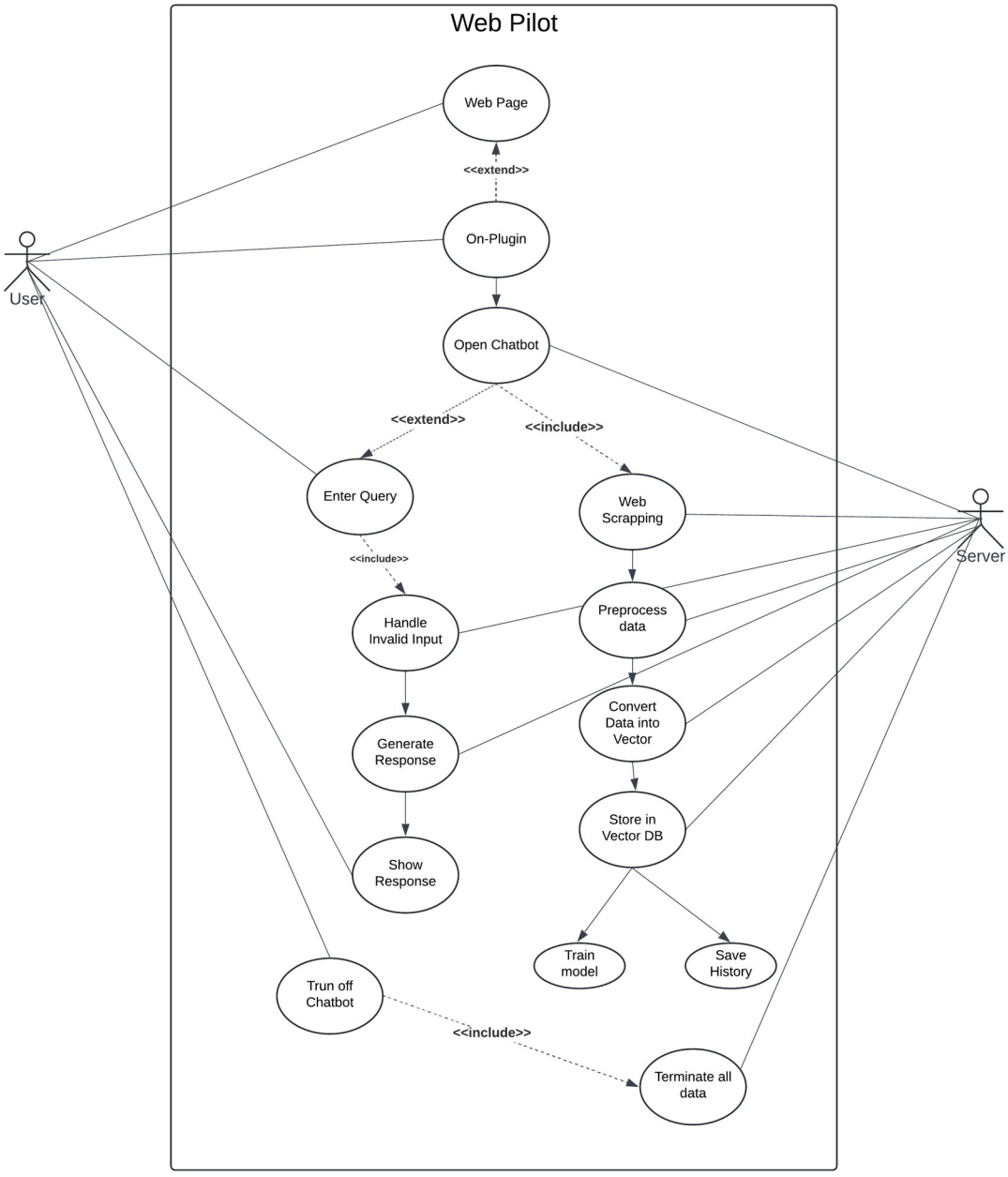


Figure 3.1: Use Case Diagram

3.5 Use Cases Description

Fully detailed use cases are thoroughly elaborated.

3.5.1 Use case: Web Page

Table 3.17 : Web Page

Use-case id	UC01
Name	Web Page
Actors	User
Pre-condition	User has internet access and a browser.
Post-condition	The web page is displayed on the user's browser.
Main Success Scenario: User navigates to the web page and the page loads successfully**.**	
Actor action	**System Response**
1. User enters the URL or clicks a link to the web page.	1.1 Web page loads and is displayed.
Alternative Path: 1. User encounters a 404 error. 2. User gets a timeout error.	

3.5.2 Use case: On Plugin

Table 3.18: On Plugin

Use-case id	UC02
Name	On Plugin
Actors	User
Pre-condition	User is on the web page and plugin is available.
Post-condition	Chatbot plugin is activated and visible.
Main Success Scenario: Plugin loads and the chatbot interface appears.	
Actor action	**System Response**
1. User clicks on the plugin icon or button.	1.1 Plugin loads and the chatbot interface

	appears.
Alternative Path: 1. Plugin fails to load due to browser compatibility issues. 2. Plugin requires additional permissions.	

3.5.3 Use case: Chatbot Open

Table 3.19: Chatbot Open

Use-case id	UC03
Name	Chatbot open
Actors	User
Pre-condition	Plugin is activated.
Post-condition	Chatbot interface is ready for user interaction.
Main Success Scenario: Chatbot interface loads successfully.	
Actor action	**System Response**
1. None (automatically happens after plugin activation).	1.1 Chatbot interface is displayed and prompts user to enter a query.
Alternative Path: 1. Chatbot interface fails to load due to network issues. 2. Chatbot is blocked by browser security settings.	

3.5.4 Use case: Enter Query

Table 3.20: Enter user query

Use-case id	UC04
Name	Enter Query
Actors	User
Pre-condition	Chatbot interface is open.
Post-condition	Query is submitted for processing.
Main Success Scenario: User enters a query via text or voice and it is submitted.	
Actor action	**System Response**

1. User types a query in the text box or speaks into the microphone.	1.1 Query is received and acknowledged by the system.
Alternative Path: 1. Voice input is not recognized properly. 2. Text input fails to submit due to network issues.	

3.5.5 Use case: Handle Invalid Input

Table 3.21: Handle Invalid User input

Use-case id	UC05
Name	Handle Invalid Input
Actors	User, Server
Pre-condition	User has entered a query into the chatbot interface.
Post-condition	User is informed about the invalid input and prompted to enter a valid query.
Main Success Scenario: 1. User enters an invalid query. 2. Server detects the invalid input. 3. Server informs the user and prompts for a new query.	
Actor action	**System Response**
1. User submits an invalid query.	1.1 Server detects the invalid query. 1.2 Server sends an error message and prompt for a valid input to the chatbot interface.
Alternative Path: 1. User submits a partially correct query. 2. Server provides suggestions to correct the query.	

3.5.6 Use case: Response Generate

Table 3.22: Response Generate

Use-case id	UC06
Name	Response Generate

Actors	User
Pre-condition	Query is entered and submitted.
Post-condition	System generates a response to the query.
Main Success Scenario: System processes the query and generates an appropriate response.	
Actor action	**System Response**
1. None (automated process).	1.1 System generates a response based on the query data.
Alternative Path: 1. System fails to generate a response due to data processing errors. 2. Response generation takes too long due to server load.	

3.5.7 Use case: Show Answer

Table 3.23: Show LLM Answer

Use-case id	UC07
Name	Show Answer
Actors	User
Pre-condition	Response is generated.
Post-condition	Answer is displayed to the user.
Main Success Scenario: System displays the generated answer to the user's query.	
Actor action	**System Response**
1. None (automated process).	1.1 Generated answer is shown in the chatbot interface.
Alternative Path: 1. Answer display fails due to interface issues. 2. Answer is not relevant and user needs to re-enter the query. 3. Question is beyond the scope of the current data, but I can provide answer. 4. Your question is out of box / current data.	

3.5.8 Use case: Off Chatbot

Table 3.24: Switch off chatbot

Use-case id	UC08	
Name	Off Chatbot	
Actors	User	
Pre-condition	Chatbot is open.	
Post-condition	Chatbot is closed and session is terminated.	
Main Success Scenario: User closes the chatbot.		
Actor action		**System Response**
1. User clicks on the close button of the chatbot.		1.1 Chatbot interface closes.
Alternative Path: 1. Chatbot fails to close due to browser issues. 2. Session data is not terminated properly.		

3.5.9 Use case: Terminate Data

Table 3.25: Terminate data

Use-case id	UC09	
Name	Terminate Data	
Actors	User,Server	
Pre-condition	Chatbot is closed.	
Post-condition	User session data is cleared.	
Main Success Scenario: System clears all session-related data.		
Actor action	**System Response**	
1. User closes the chat-bot.	1.1 Server terminates the session and clears all related data.	
Alternative Path: 1. Data termination fails due to server error. 2. Session data is not terminated properly. 3. Data is not properly cleared due to session timeout		

3.5.10 Use case: Text Scrape

Table 3.26: Text Scrapping

Use-case id	UC10	
Name	Text Scrape	
Actors	Server	
Pre-condition	Query is entered and requires external data.	
Post-condition	Relevant data is scraped from the web.	
Main Success Scenario: System scrapes relevant data from external sources based on the query.		
Actor action		**System Response**
1. None (automated process).		1.1 Data is scraped and prepared for processing.
Alternative Path: 1. Text scraping fails due to restrictions. 2. Data scraped is not relevant or incomplete.		

3.5.11 Use case: Preprocess Data

Table 3.27: Preprocess Data

Use-case id	UC11	
Name	Preprocess Data	
Actors	Server	
Pre-condition	Data is scraped from the source	
Post-condition	Data is cleaned and formatted for further processing.	
Main Success Scenario: System preprocesses the scraped data.		
Actor action		**System Response**
1. None (automated process).		1.1 Data is preprocessed and ready for vector conversion.
Alternative Path: 1. Data preprocessing fails due to formatting issues. 2. Preprocessed data is incomplete or incorrect.		

3.5.12 Use case: Convert Data into Vector

Table 3.28: Convert data into vectors

Use-case id	UC12
Name	Convert Data into Vector
Actors	Server
Pre-condition	Data is preprocessed.
Post-condition	Data is converted into vector format.
Main Success Scenario: System converts preprocessed data into vector format.	
Actor action	**System Response**
1. None (automated process).	1.1 Data is converted into vector format.
Alternative Path: 1. Data conversion fails due to incompatible data format. 2. Converted vectors are incorrect or corrupted.	

3.5.13 Use case: Store in Vector DB

Table 3.29: Store in Vector DB

Use-case id	UC13
Name	Store in Vector DB
Actors	Server
Pre-condition	Data is converted into vector format.
Post-condition	Vector data is stored in the database.
Main Success Scenario: System stores the vectorized data in the database.	
Actor action	**System Response**
1. None (automated process).	1.1 Vector data is saved in the database.
Alternative Path: 1. Database storage fails due to server issues. 2. Data is not stored correctly and needs reprocessing.	

3.5.14 Use case: Model Train

Table 3.30: Model Training

Use-case id	UC14
Name	Model train

Actors	Server
Pre-condition	Vector data is stored in the database.
Post-condition	Model is trained using the vector data.
Main Success Scenario: System trains the model with the stored vector data.	
Actor action	**System Response**
1. None (automated process).	1.1 Model is trained and ready for use.
Alternative Path: 1. Model training fails due to insufficient data. 2. Training is interrupted by server issues.	

3.5.15 Use case: Save History

Table 3.31: Save History

Use-case id	UC15
Name	Save history
Actors	Server
Pre-condition	User interaction has occurred.
Post-condition	Interaction history is saved.
Main Success Scenario: System saves the user's interaction history.	
Actor action	**System Response**
1. None (automated process).	1.1 Interaction history is saved in the database.
Alternative Path: 1. History saving fails due to database issues. 2. Interaction data is incomplete and needs reprocessing.	

3.6. Summary

Chapter 3, "Requirements Specification," outlines the detailed requirements for the Web Pilot project, focusing on both functional and non-functional aspects. It defines the core functionalities of the platform, including user interaction, chatting, Content related responses and cost estimation tools.

CHAPTER 4

SYSTEM MODELING

4.1 Introduction

In this, we engage in the modeling of the system currently in development. System modeling, the process of creating abstract representations of a system, is employed, where each model provides a distinct view or perspective of the system. The aim of system modeling is to represent the system graphically, utilizing notations from the Unified Modeling Language (UML). This graphical representation allows viewers and analysts to comprehend the system's functionality and the approaches used in communication with customers. System design reveals the system's features and the environment in which it will operate.

4.2 (4+1) View Model

The 4+1 View Model is a crucial architectural approach designed to tackle the inherent complexities of communicating software architecture effectively. As depicted in Figure 4.1, this model provides a structured framework for visualizing and understanding the multifaceted nature of a software system. It breaks down the system's architecture into five distinct, yet interconnected, views, each serving a specific purpose and addressing the concerns of different stakeholders. It provides a set of scenarios or use cases that demonstrate how the system is used by different actors. This view ensures that the architecture aligns with the system's requirements and that all stakeholders have a shared understanding of the system's behavior. The 4+1 View Model, therefore, enables a holistic and comprehensive understanding of software architecture, facilitating effective communication and collaboration among all stakeholders.

1. Logical View: Illustrated by a Class Diagram.
2. Process View: Represented through Activity and Sequence Diagrams.
3. Development View: Depicted by a Component Diagram.
4. Physical View: Presented via a Deployment Diagram

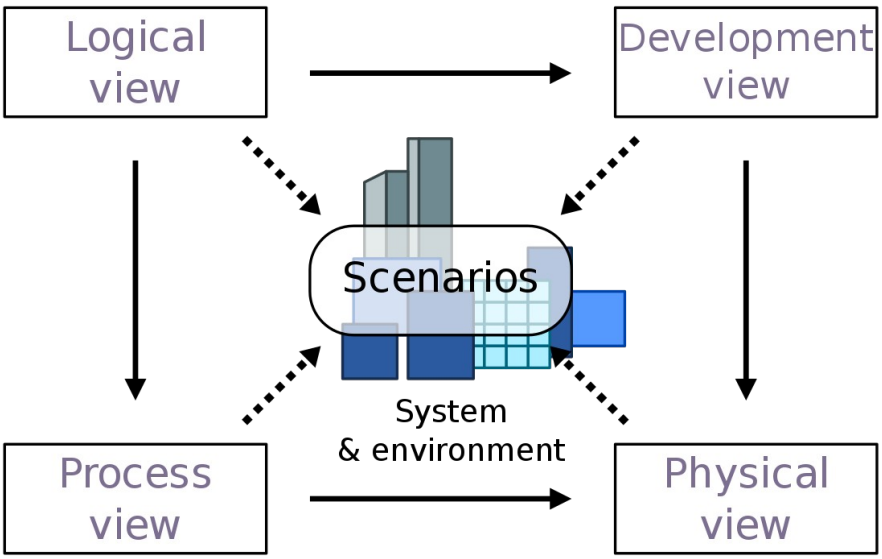


Figure 4.1: 4+1 View Model

4.2.1 Logical View

In logical view, we show static models view and its connection with functionalities of our projects. For this purpose, we use Class diagram.

4.2.1.1 UML Class Diagram

The class diagram stands as a cornerstone in software development and design, providing a visual representation of a system's static structure. Developers leverage this diagram to meticulously define the building blocks of their software, encompassing classes, their inherent attributes, the methods that dictate their behavior, and the intricate relationships that connect them. As depicted in Figure 4.2, a well-constructed class diagram effectively outlines each class's properties (attributes) and operations (methods), while simultaneously illustrating how these classes interact through associations, aggregations, compositions, and inheritance. By providing a blueprint of the system's object-oriented structure, the class diagram promotes effective communication among team members, guides implementation, and serves as a crucial artifact for code generation, testing, and maintenance, ultimately contributing to the creation of robust and well-organized software systems.

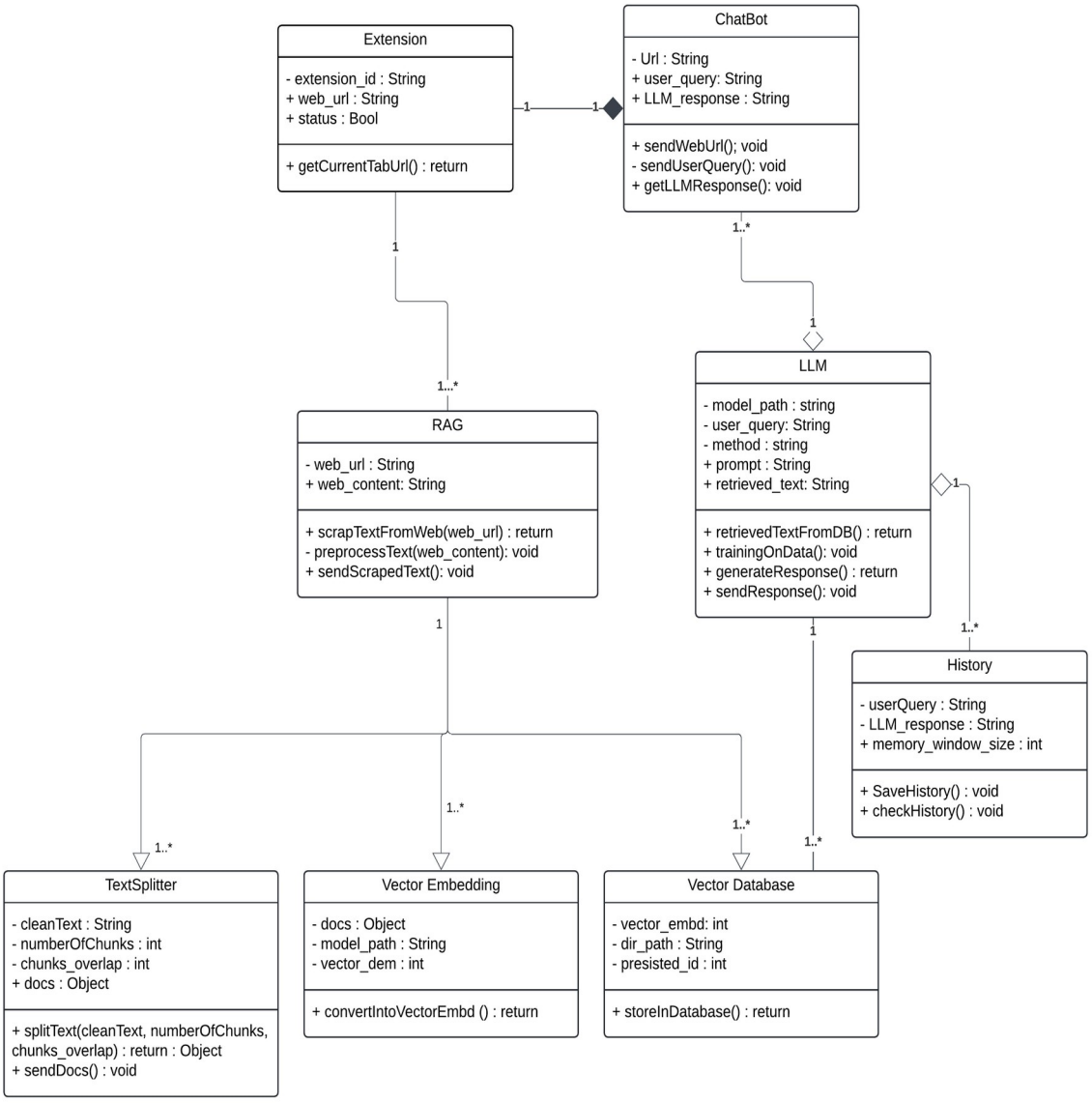


Figure 4.2: Class Diagram

4.2.2 Process View

The process view, only the Activity Diagram and Sequence Diagram is covered in our system because there is no hardware involvement, so the State Machine Diagram is not included. The activity diagram will display the whole system's flow in terms of the users.

4.2.2.1 UML Activity Diagram

Figure 4.3 illustrates the workflow and activities of user within the system, providing a visual representation of their interactions and tasks.

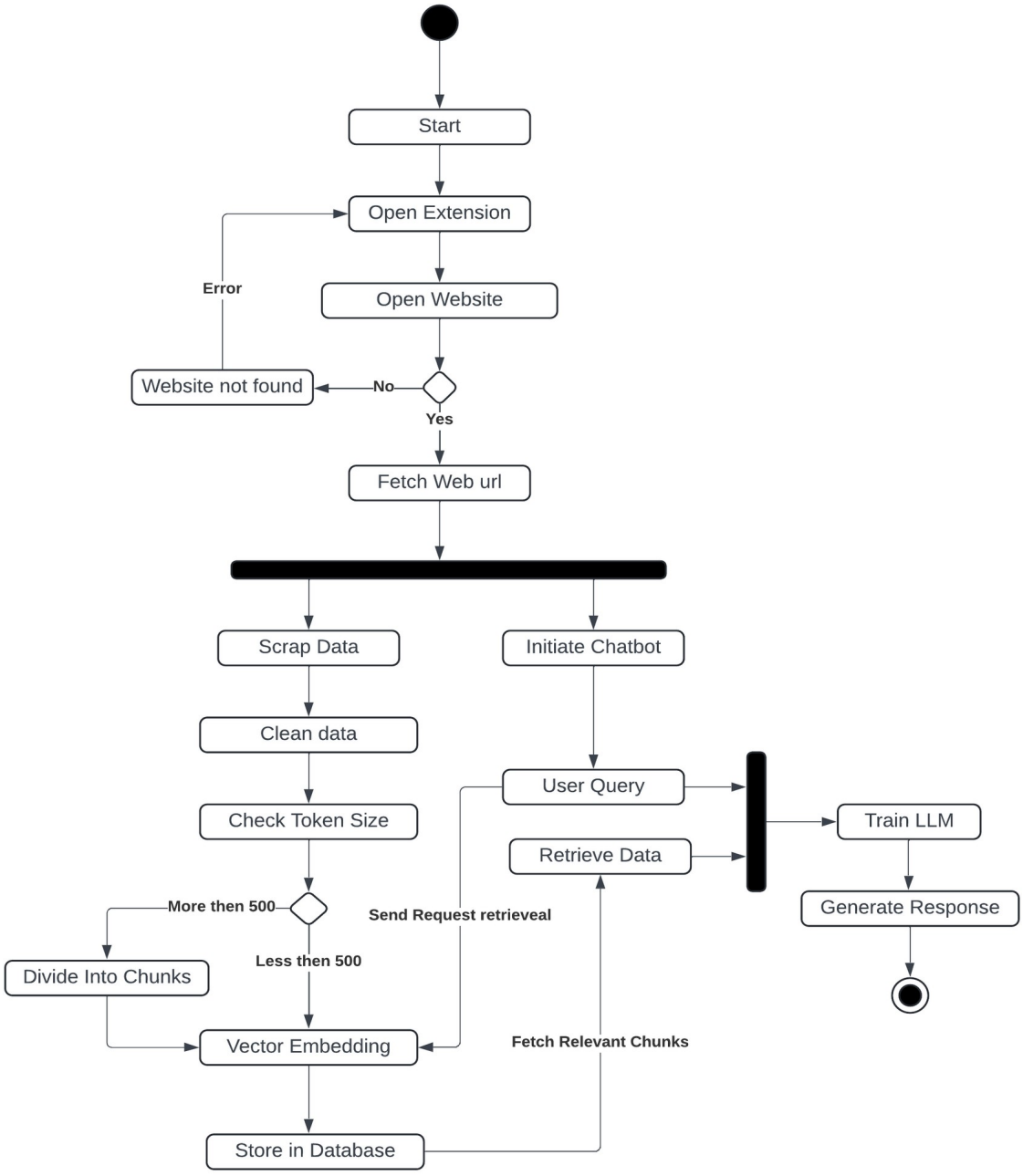


Figure 4.3: UML Activity diagram

4.2.3 Sequence Diagram

Sequence diagrams show how different software objects interact over time. They use the vertical axis to display when messages are sent, clearly illustrating the order of interactions in a software system.

4.2.3.1 User Sequence Diagram

This sequence diagram will illustrate the user interaction with the system, detailing how messages and responses flow throughout the system and how the user receives these responses.

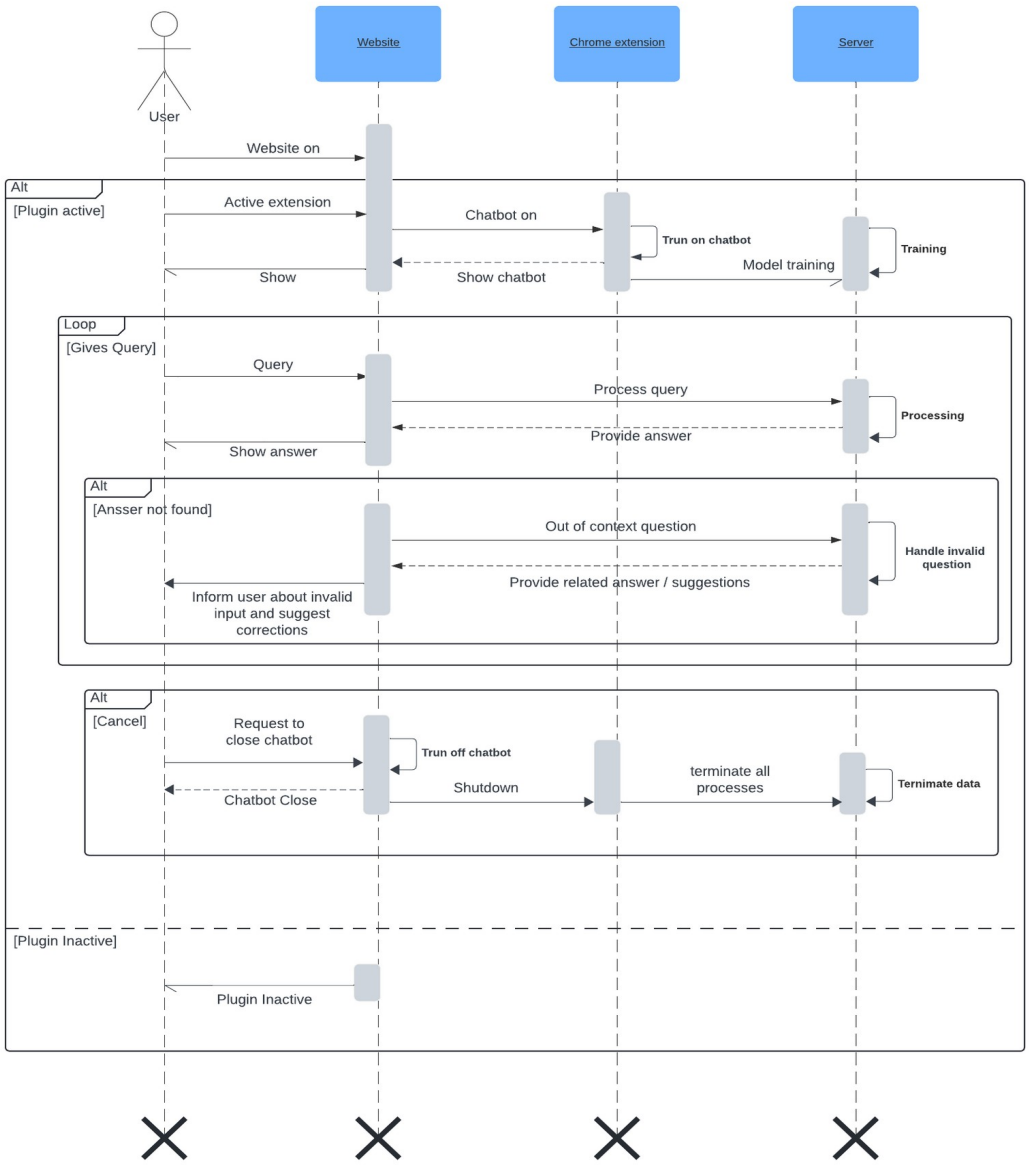


Figure 4.4: User Sequence Diagram

4.2.3.2 Response Generate Sequence Diagram

This sequence diagram provides a detailed representation of the server's operational workflow. It outlines the step-by-step process of how the server handles user interactions, processes incoming responses, and manages data collection for enhancing the performance of the large language model (LLM). Additionally, it highlights the implementation of Retrieval-Augmented Generation (RAG) techniques, which enable the system to fetch relevant information from external sources before generating responses. Furthermore, the diagram demonstrates how the server integrates web data to produce well-informed and contextually relevant answers for users.

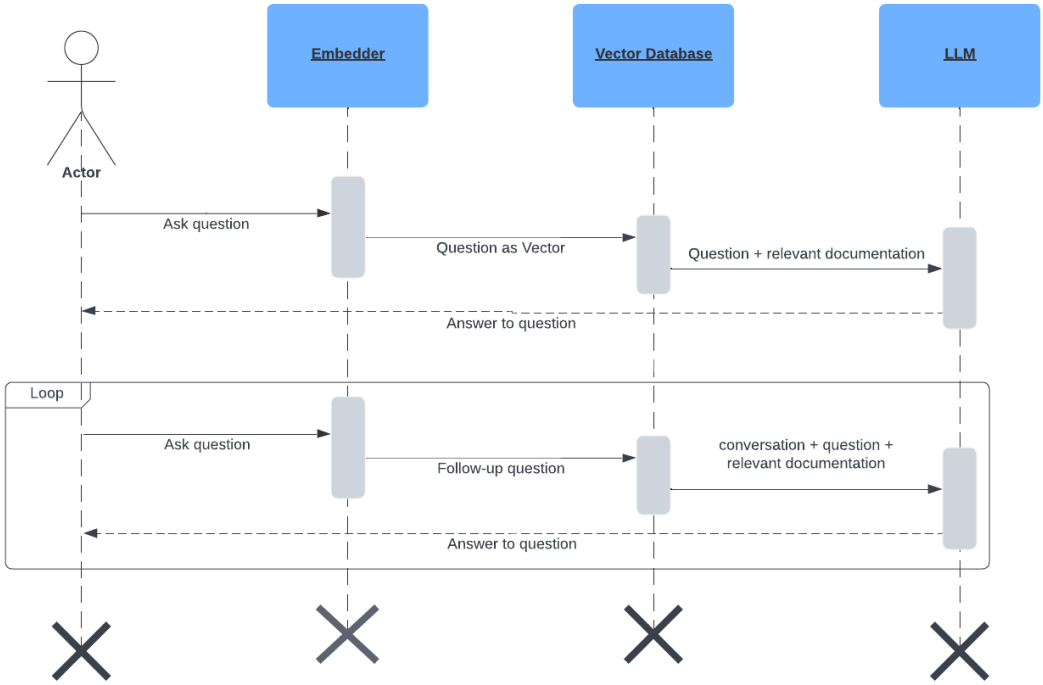


Figure 4.5: Response Generation Sequence Diagram

4.2.4 Development View

The development of our system which has component diagram is given below.

4.2.4.1 UML Component Diagram

Figure 4.6 presents a comprehensive visual representation of the various components that make up the system and how they interact with one another. This diagram serves as an architectural overview, illustrating the structure, organization, and interconnections between different system elements. By examining this figure, one can gain a clear understanding of the roles and relationships of each component, as well as how data and processes flow throughout the system to achieve the desired functionality.

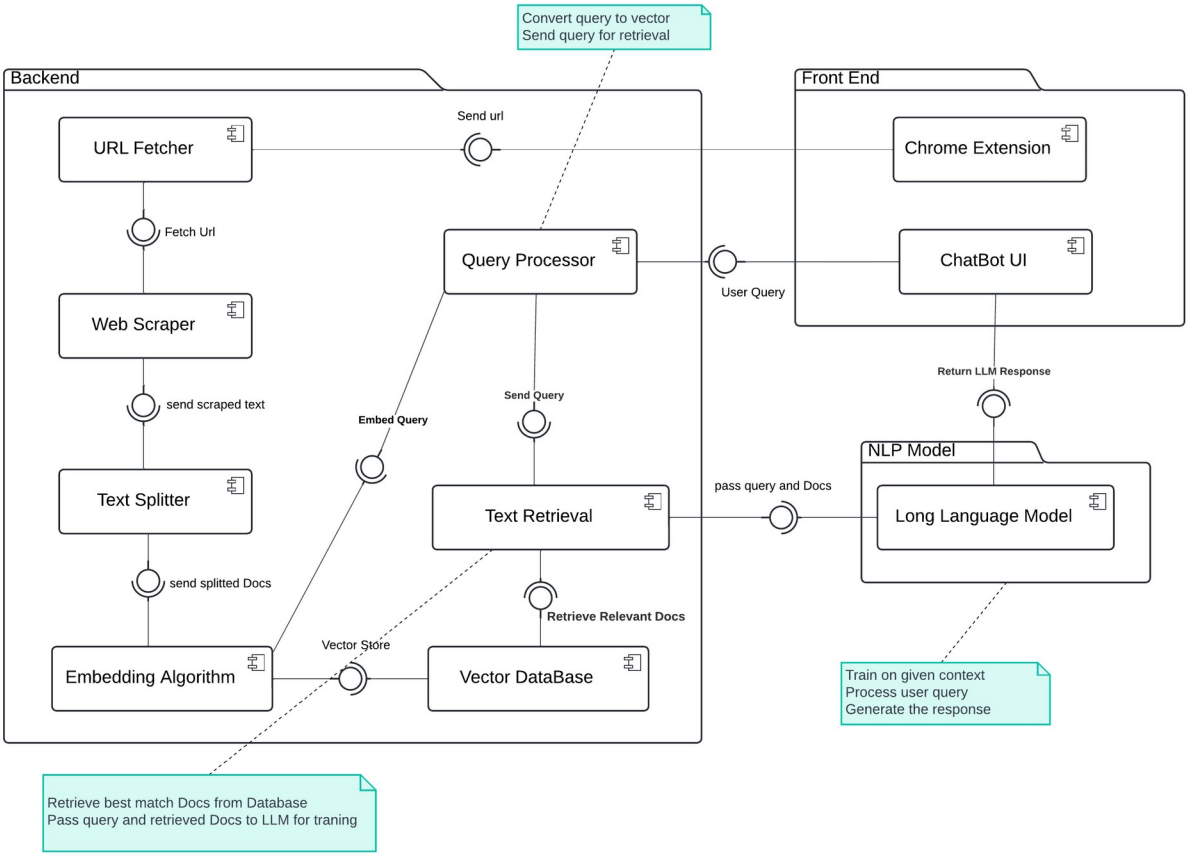


Figure 4.6: Component Diagram

4.2.5 Physical View

Given below is the physical view of our system:

4.2.5.1 UML Deployment Diagram

Figure 4.7 offers a detailed visual representation of the system's deployment architecture, showcasing how various components are distributed across different nodes, servers, or environments. This diagram helps in understanding the structural layout of the deployment, including how different elements interact, communicate, and function within their respective environments. By analyzing this figure, one can gain valuable insights into the system’s deployment configuration, scalability, and resource allocation, ensuring an efficient and well-organized implementation.

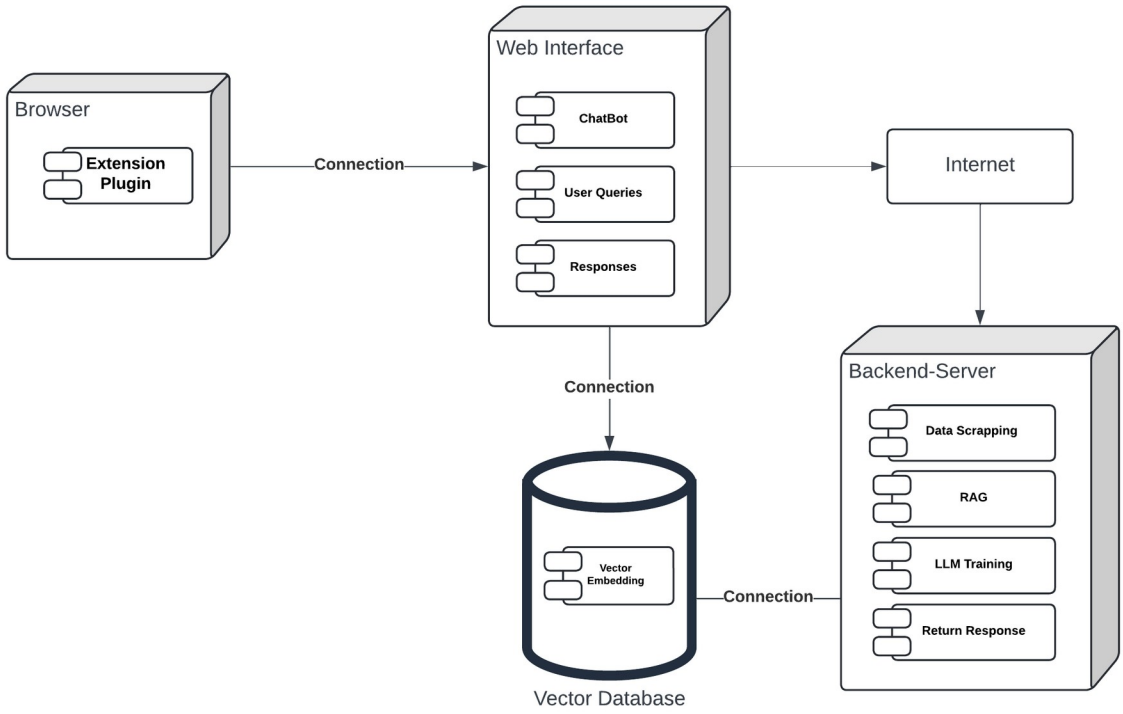


Figure 4.7: Deployment Diagram

4.2.6 System Prototype

The prototyping of our system in Figma is given below:

4.2.6.1 Initial Page

In this prototype, when the user opens our extension, it will fetch the link displayed at the top of the chatbot interface. Initially, it will display some suggestion messages, as seen in the figure 4.8.

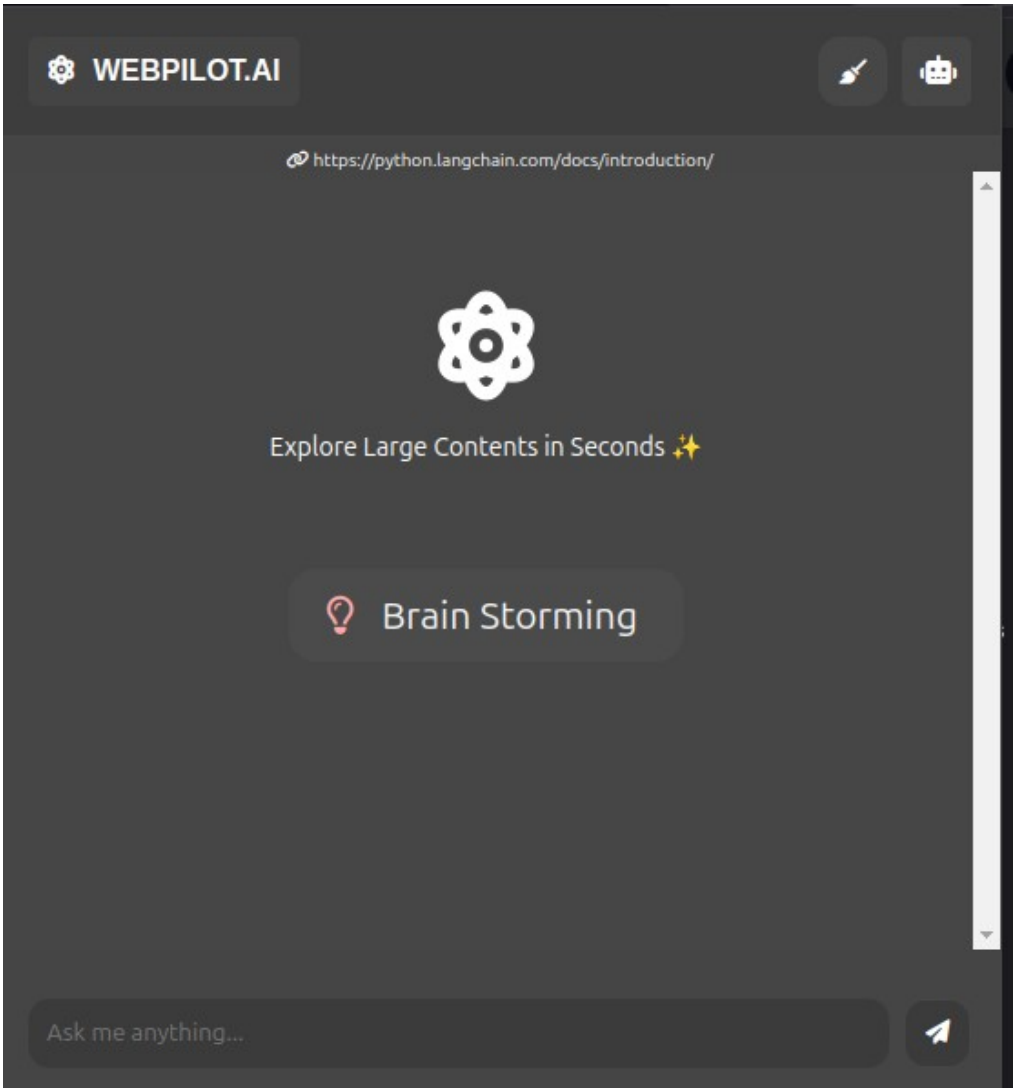


Figure 4.8: Web Pilot Initial Page

4.2.6.2 Home Page

Once you're on the starting page, simply start chatting with “Web Pilot”. You can ask any questions or share any queries you have about the webpage youre viewing, and “Web Pilot” will give you accurate answers. You can clear chat, see progress and LLM models we used. It's a convenient way to get information and assistance while browsing as seen in the figure 4.9.

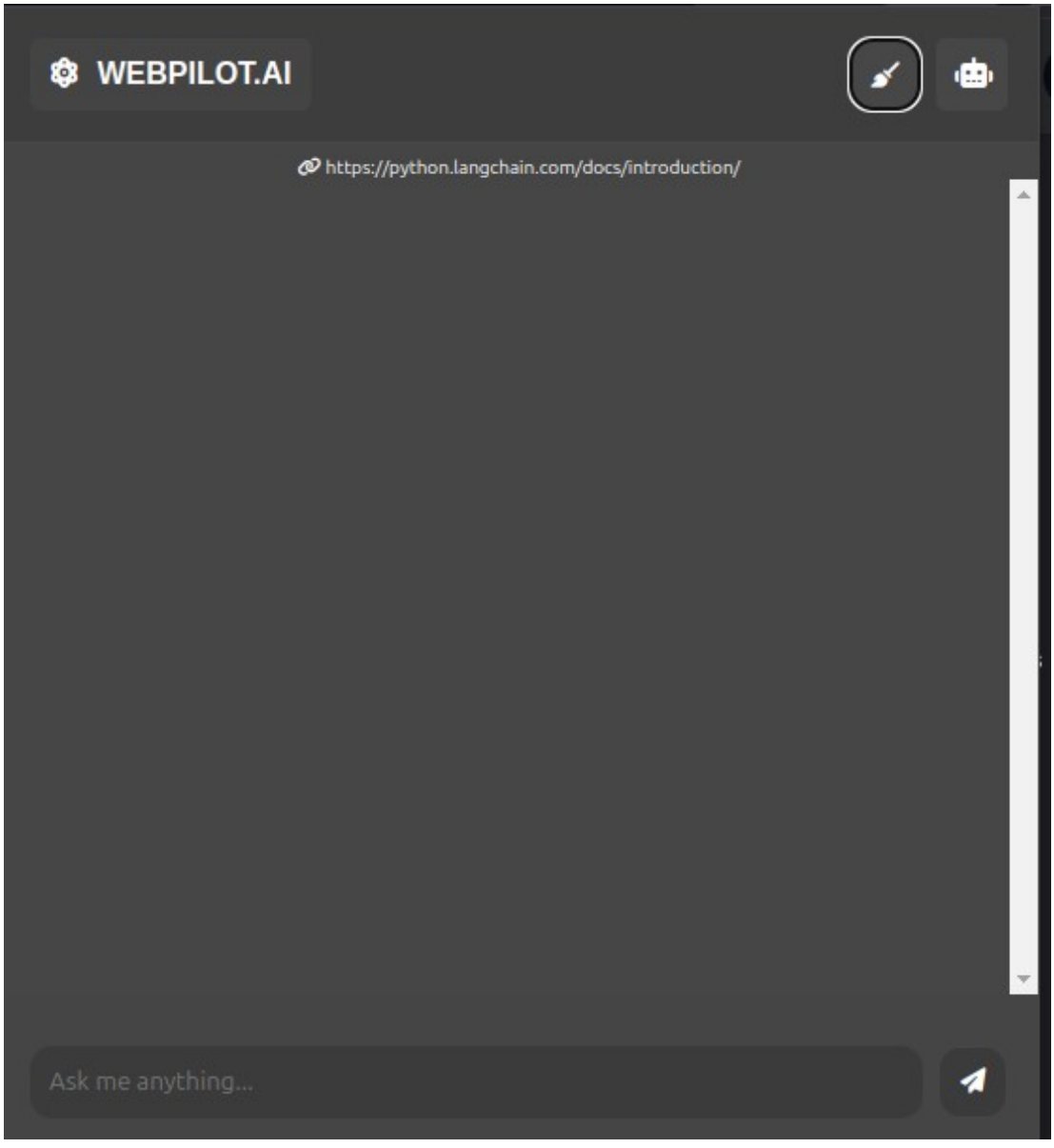


Figure 4.9: Web Pilot Home Page

4.2.6.3 Chat Page

On the chat page you can ask questions about any web content you open in your browser. As shown in Figure 4.10, the chatbot will engage in a conversation with you, similar to a human, and help you navigate the content of the page you're viewing. The URL of that page will be displayed in front of the content.

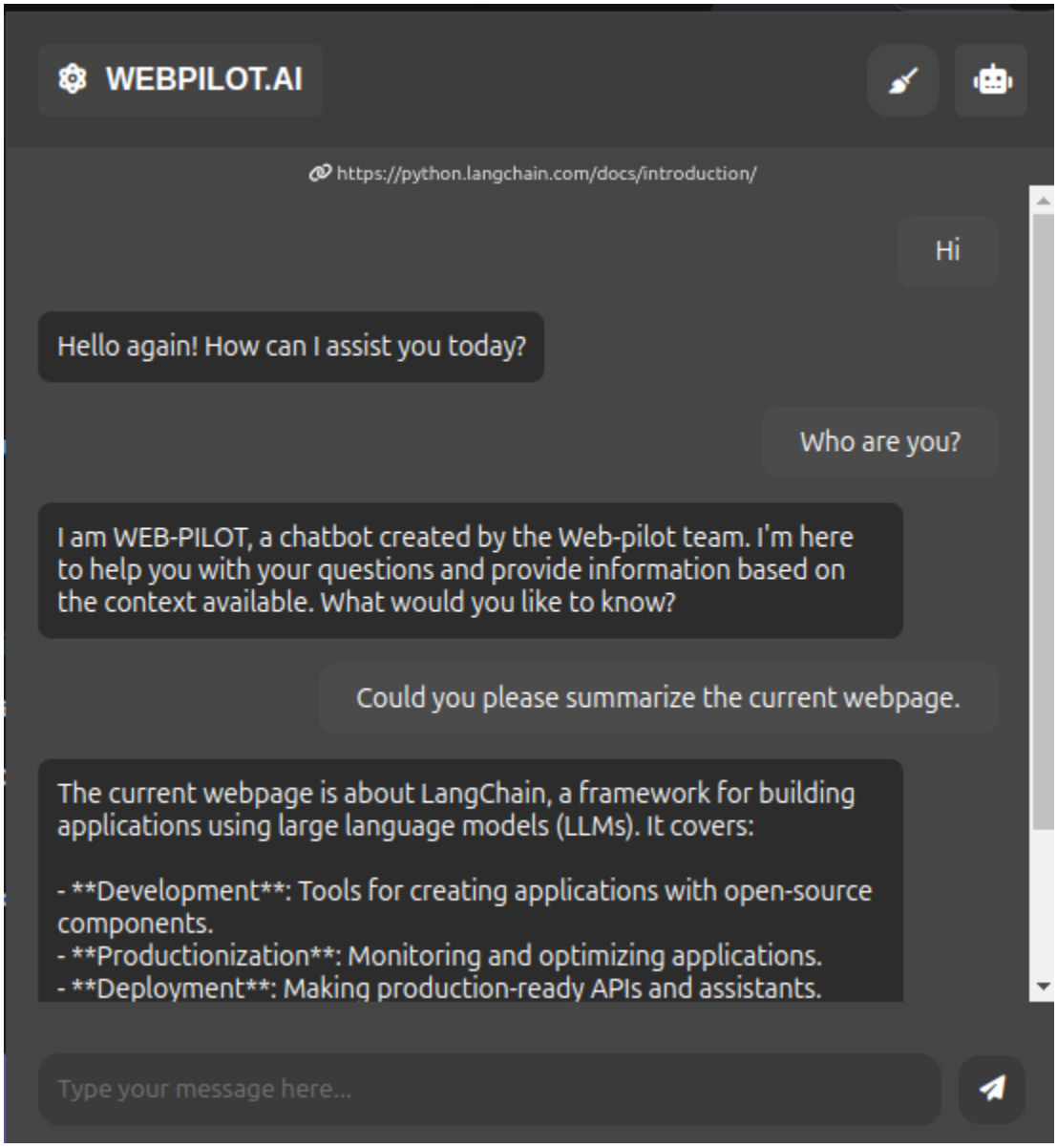


Figure 4.10: Web Pilot Chat Page

4.2.6.3 Model Information

By clicked you model Icon you can see information about LLM chatbot Generative AI models. You can see we use three LLMs models in Web-Pilot

1. GPT-4o-mini
2. Mistral-7B-instruct
3. Llama-3.1-instruct

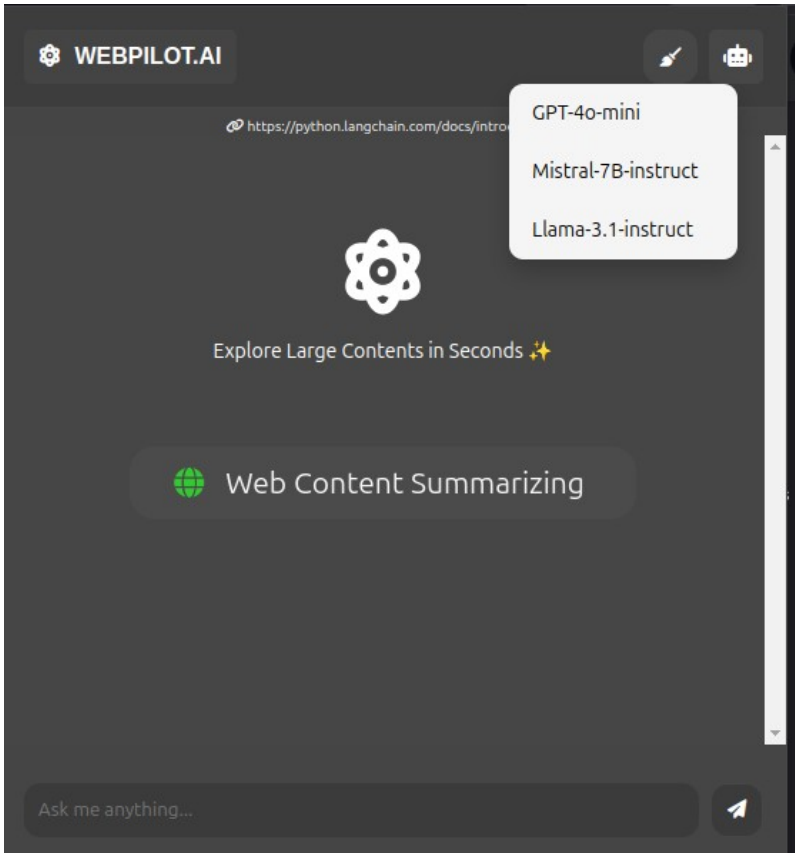


Figure 4.11: Model information

4.3. Summary

Chapter 4, "System Modeling" outlines the detailed structure of for the Webpilot project, focusing on both Universal Modeling Language and system interactions. It defines the how user will intract with system according to logical view, physical view and other UML diagrams.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

In this chapter, we will discuss the implementation details of each module including those modules that involves libraries, frameworks, models and API we used in our project.

5.2 Modules of the System

The modules of our system are discussed in the following.

5.2.2 Browser Extension

Web-Pilot Browser extension help users interact with websites, web pages, and PDFs more easily, without the need to copy and paste URLs into specific LLM AI assistant to get answers. Users simply activate the browser extension on the webpage, YouTube video, or PDF document they wish to chat about, summarize, or ask questions regarding. This enhances accessibility and allows users to turn the plugin on and off as needed, providing a better user experience and saving time by eliminating the need to switch between windows and copy links.

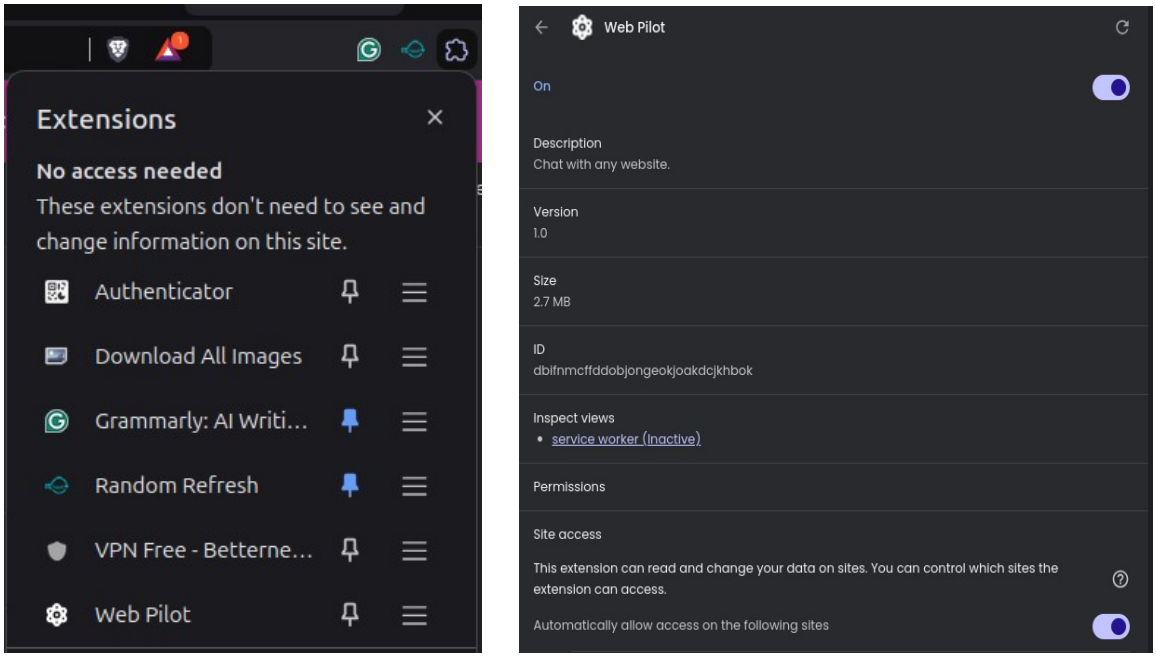


Figure 5.1: Chrome Extension

5.2.1 Conversation Panel

The conversational panel module provides an organized interface for viewing your conversations with the AI language model. Users can scroll through the entire conversation, clear the chat, and enjoy unlimited interactions. This module allows users to engage with our system while delivering an exceptional user experience through streaming text, creating a typing flow that feels natural. When users open the application for the first time, animated suggestions appear, showcasing the capabilities of our system. This feature helps users understand the purpose of the application effectively.

5.2.3 Content Extraction

When the user clicks on the WebPilot Chrome extension, it will immediately extract all text from the content of the currently open tab, whether it's a website, a YouTube video, or a PDF. The user must activate and open the WebPilot extension to enable this feature. Additionally, the extension can retrieve the URL of the current tab. Content extraction is crucial for training models using this text data.

5.2.4 Data Preprocessing and Parsing

After extracting text content from webpages, YouTube, or PDF documents, we need to preprocess and clean the text. This module involves removing unnecessary information separate the headings, tables, lists bullet points in data, filtering out URLs, verifying the correct format of URLs, and converting them to their proper format. We should also remove all empty lines and significant gaps in the text. Once the text is cleaned and parsed, it becomes more suitable for training language models, allowing them to better understand the content. Parsing helps convert the text into a streamlined format, making it easier to maintain as you can see in fig 5.3.

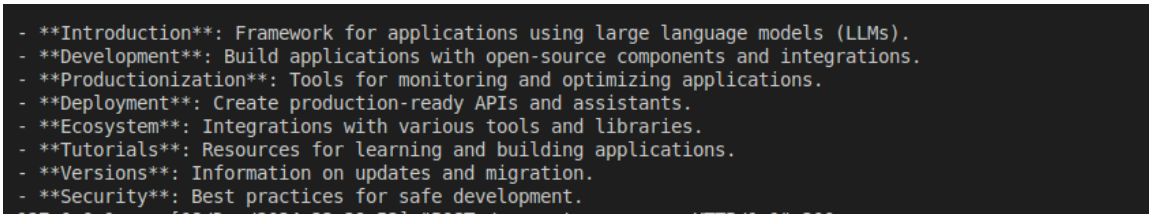


Figure 5.2: Data Preprocessing and Parsing

5.2.5 Retrieval-Augmented Generation (RAG)

In our system, RAG (Retrieval-Augmented Generation) helps us train our LLM (Large Language Model) quickly and in a contextual manner. "Contextual" means that the model can temporarily train on a given context, and then when a new context is provided, it trains on the latest context and disregards the previous one. RAG enables us to provide quick responses to users. Below, we will discuss several steps we utilize from RAG.

5.2.5.1 Create Documents

When a user clicks on the Web-Pilot Chrome extension, it intelligently extracts all the text from the currently open webpage, YouTube transcripts, and PDF files. This text is then compiled into a well-structured document, streamlining the organization of data for efficient processing. This step serves as the foundation of the (RAG) pipeline, ensuring that the text data is loaded and ready for further analysis and interactions.

5.2.5.2 Split Documents

Once the text data is compiled into documents, it is segmented into chunks, each containing approximately 1500 words with an overlap of 40 words between consecutive chunks. This segmentation ensures consistency and coherence across the text, enabling better organization of the content. The overlapping words help maintain context between chunks, which is crucial for accurate text embeddings and efficient downstream processing in the RAG pipeline.

5.2.5.3 Text Embeddings

In this step, the segmented text chunks undergo a transformation into high-dimensional vector representations known as embeddings. These embeddings capture the semantic meaning of the text, enabling the system to understand the relationships between different chunks. By converting textual data into numerical representations, this process lays the groundwork for efficient search, retrieval, and matching of relevant information in the RAG pipeline. The embeddings are a critical component for enhancing the chatbot ability to deliver accurate and contextually relevant responses.

5.2.5.4 Vector Database

We use FAISS vector database which is most efficient and fast vector database to store embeddings. Once the text chunks are converted into embeddings, they are stored in a vector database. This specialized database is designed to efficiently manage and search high-dimensional vectors. It enables fast retrieval of semantically similar embeddings based on user queries. By organizing and indexing the embeddings, the vector database plays a crucial role in powering the RAG pipeline, ensuring quick and accurate access to relevant information from the processed content.

5.2.5.5 Chat Memory Management

In this step, the system manages the chat history and remembers previous conversations. This allows the chatbot to keep track of context across multiple interactions, so it can provide more personalized and coherent responses. By maintaining a memory of previous chats, the system can build on earlier interactions and understand the user's ongoing needs and preferences, making the conversation smoother and more useful. In this RAG module we use conversation buffer memory which help to maintain the last five conversation history as you can see in code fig 5.2.

```
# Set up window memory for conversation
self.window_mem = ConversationBufferWindowMemory(
    k=3,  # Number of messages to remember
    memory_key='chat_history',
    return_messages=False,
    human_prefix='Question',
    ai_prefix='Answer',
    input_key='question',
    verbose=False
)
```

Figure 5.3: Chat memory

5.2.6 Large Language Model Integration

In this module, we integrate the OpenAI Large Language Model, GPT-4o mini, which is highly efficient and fast, resulting in fewer hallucination errors. This model works alongside the user's query and a vector database to retrieve relevant information stored within. By

doing this, the LLM can concentrate on text that pertains specifically to the user's question. The LLM is then utilized to provide answers, summaries, and perform various tasks. When a user asks a question, the retrieval chain selects the top 50 most relevant text chunks from the database and uses them to train the LLM for a context-aware response. Here the code example to import LLM model.

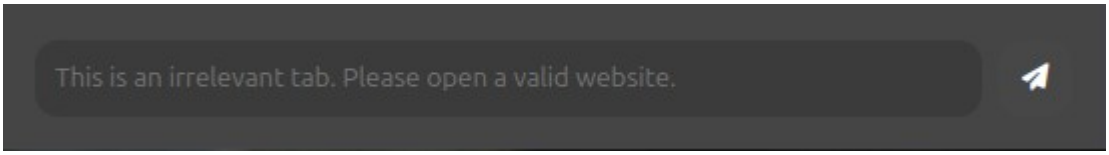
```
self.gpt_llm = ChatOpenAI(
    model='gpt-4o-mini',
    temperature=0.1,
    max_tokens=500,
    stop_sequences=self.filter,
)
```

Figure 5.4: LLM Integration

5.2.8 Error Handling

In this module, we will address various types of errors that can occur while using our chatbot and implement solutions for these issues. We have identified several common errors:

1. The user must open a webpage, YouTube video, or PDF content before launching the Chrome extension; otherwise, they will not be able to input queries to the chatbot.



2. If a user opens a chatbot within a PDF file that contains no text or is a scanned PDF where text cannot be extracted, an error will occur. We handle this error by displaying a clear message to the user, instructing them to use PDF files that contain text.

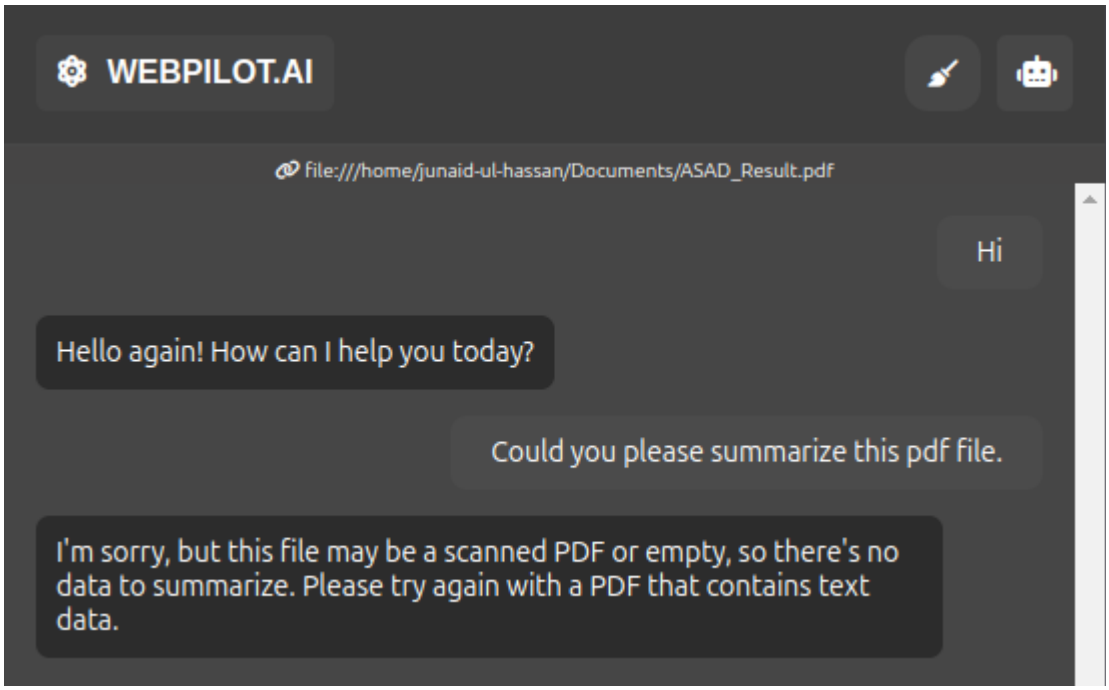


Figure 5.5: Chat with PDF

3. If a user opens a YouTube video that does not have a transcript or has a transcript in a language other than English, the chatbot will show an error message to the user.

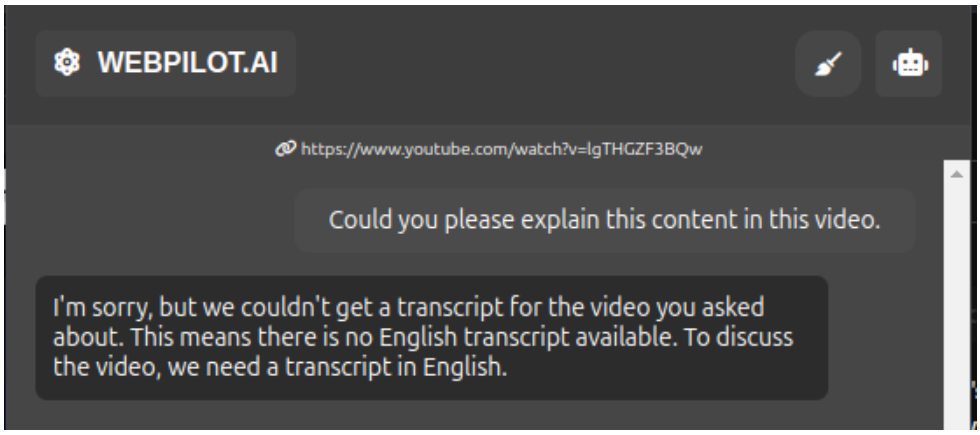


Figure 5.6: Chat with YouTube videos

These measures help ensure a smoother experience while interacting with the chatbot.

5.3 Frameworks

In our project, we used several frameworks to develop an effective solution.

5.3.1 Chrome Extension Development

In our system, we develop Chrome extensions using a comprehensive framework. The extensions must be built using HTML, CSS, and JavaScript for the frontend. For the backend, any suitable web development framework can be utilized, provided it aligns with the project's requirements and guidelines. There some important steps in develop efficient chrome extension.

5.3.1.1 Load Unpacked Extension

To test and debug a Chrome extension during the development process, it is essential to load it as an "Unpacked Extension" in Chrome. This allows developers to work with their extension in a local environment before publishing it to the Chrome Web Store.
The steps to load an unpacked extension are as follows:

1. Navigate to the Chrome browser and open the **Extensions page** by typing `chrome://extensions/` in the address bar.

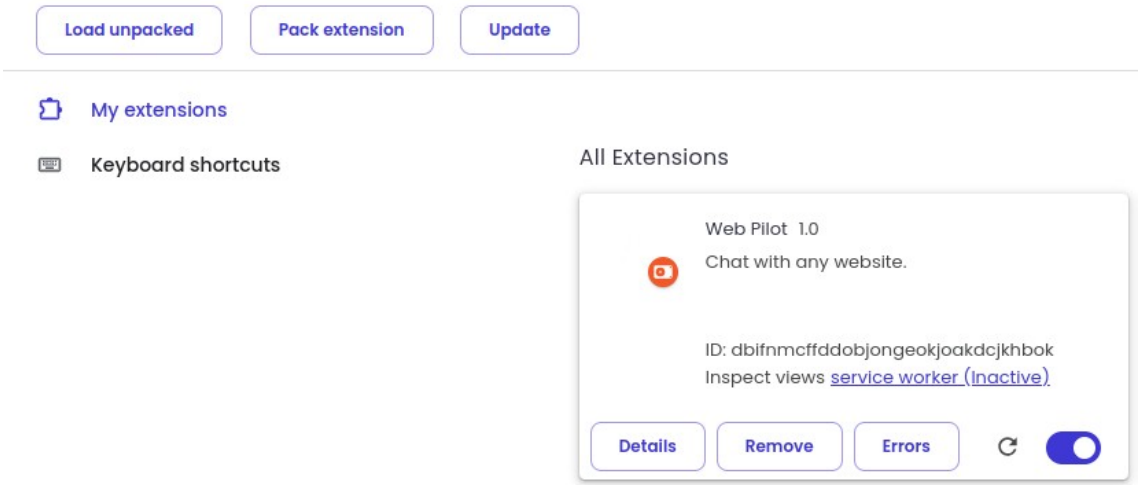
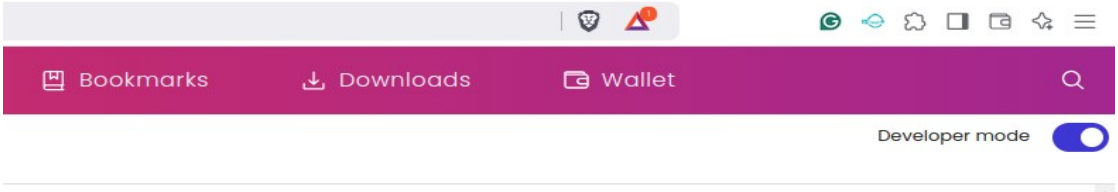
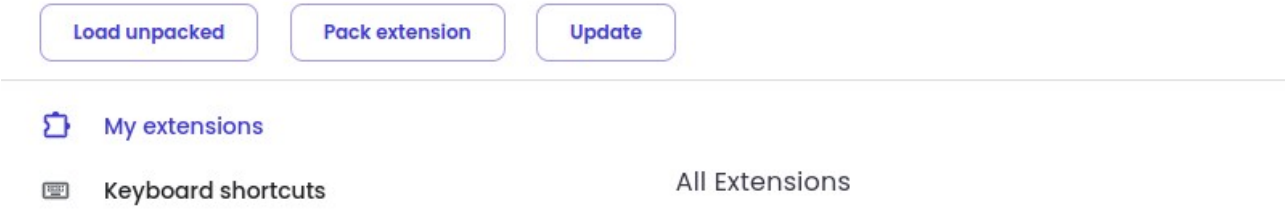


Figure 5.7: Load Upacked Extension

2. Enable **Developer mode** by toggling the switch at the top-right corner of the Extensions page.



3. Click on the **Load unpacked** button.



4. Select the folder containing your extension's files, ensuring that the `manifest.json` file is present at the root level.

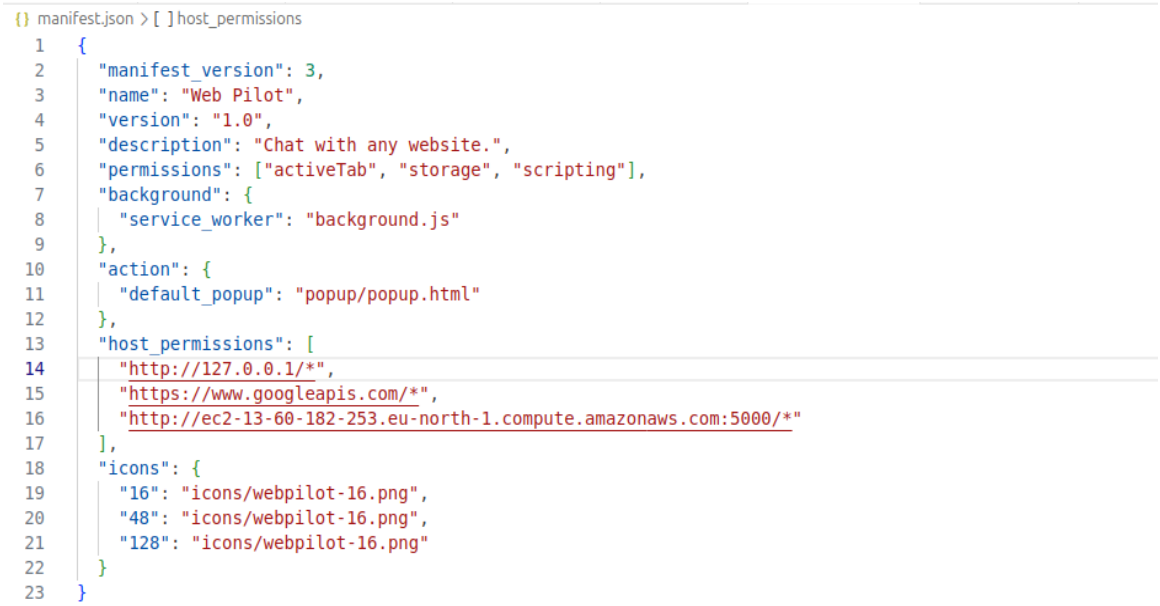


Figure 5.8: Manifest code

Once loaded, the extension will appear in the Extensions list, and any changes to the code can be updated in real time by reloading the extension. This step is crucial for testing features, debugging, and ensuring compatibility with Chrome's extension environment.

5.3.1.2 Structure an Extension Project

There is a specific structure for a Chrome extension that we should follow before uploading the extension to the Chrome browser.

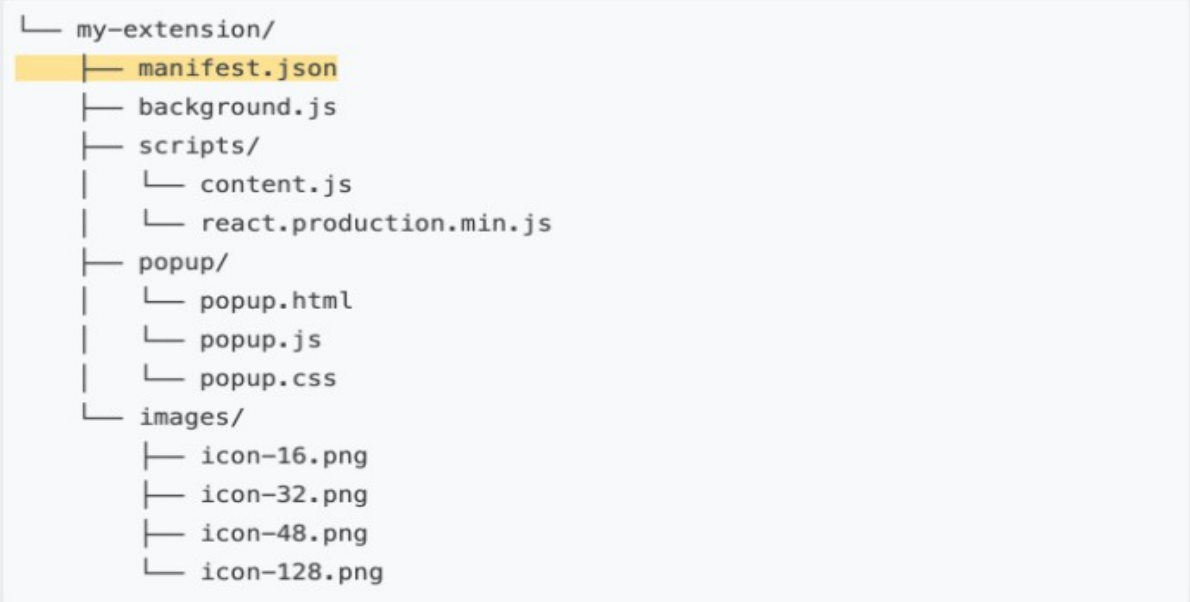


Figure 5.9: Extension file structure

5.3.2 HTML/CSS/JS

In our system, we use **HTML** to create the user interface for the Webpilot chatbot. This includes components such as the input text field, send button, clear chat button, model information button, chat container, user messages, and AI assistant messages. **CSS** is used to style the chatbot, ensuring it is visually appealing and provides a user-friendly, interactive design. **JavaScript** is implemented to manage all events, such as handling chat interactions, text streaming, and adding animations to enhance the overall user experience. Send user response backend server and get response from backend server.

5.3.3 Flask (Python Backend-Framework)

In our system, we use the Flask framework in Python as the backend server to run all generative AI code. Flask facilitates quick communication with the front-end code, which is primarily written in JavaScript. When we receive a user query in Flask, it is sent to the AI model (LLM). The query then goes through the entire processing pipeline, generating a response that is sent back to the front end for display.

5.3.5 Langchain

Our project leverages the LangChain framework, a robust Python-based toolset, to harness the power of generative AI. LangChain significantly streamlines our development process by providing comprehensive support for Large Language Model (LLM) programming. Specifically, it facilitates efficient execution of Retrieval-Augmented Generation (RAG) operations, a crucial technique for enhancing LLM accuracy with external data. Moreover, LangChain enables seamless integration with cutting-edge LLMs, including OpenAI's GPT-4o, Mistral AI, and Meta AI, allowing us to utilize the most advanced models available. By simplifying and optimizing both generative AI operations and RAG workflows, LangChain empowers us to perform high-level LLM programming with increased ease and efficiency, ultimately accelerating our project's development and enhancing its capabilities.

```
5   from langchain_community.llms import HuggingFaceHub
6   from api_token import LargeLanguageModel
7   from langchain_huggingface import HuggingFaceEndpoint
8   from langchain.chains import LLMChain, RetrievalQA
9   from langchain.prompts import PromptTemplate
10  from langchain.memory import ConversationBufferWindowMemory, ConversationBufferMemory
11  from RAG import Retrieval_Augmented_Generation
12  from langchain_openai import ChatOpenAI
13  from langchain_core.output_parsers import StrOutputParser
14  import io
15  import os
16  from PIL import Image
17  import requests
18  import openai
```

Figure 5.10: Langchain Libraries

5.3.6 Docker

In our system, we need Docker, a platform that enables developers to package applications and their dependencies into containers. This ensures consistency across different environments, making it easier for us to deploy on an AWS server without needing to install all the libraries and dependencies manually. Below is a breakdown of the key components used in the Dockerfile for our project:

5.3.6.1 Base Image

The Dockerfile starts with the `python:3.10-slim` base image, which provides a lightweight Python environment to run the application.

5.3.6.2 System Dependencies

To ensure Python packages function correctly, essential system-level libraries are installed, such as:

- libffi-dev
- libjpeg-dev
- zlib1g-dev

These dependencies are installed using `apt-get`, followed by cleanup commands to reduce image size.

5.3.6.3 Working Directory

The working directory for the container is set to `/app`, where all project files will be located and executed.

5.3.6.4 Copying Application Files

The following files and folders are copied into the container:

1. The `Backend/` folder, which contains the application's source code.
2. The `requirements.txt` file, listing Python dependencies.
3. The `.env` file for environment-specific configurations.

5.3.6.5 Python Dependencies

The `pip` command is used to install all Python dependencies specified in `requirements.txt`. Additionally, `pip` is upgraded to the latest version for compatibility.

5.3.6.6 Environment Variables

Two environment variables are set to configure the Flask application:

- `FLASK_APP=backend.py`: Specifies the main entry point for the Flask application.
- `FLASK_ENV=development`: Runs the application in development mode for debugging purposes.

5.3.6.7 Port Exposure

The application is exposed on port `5000` so it can be accessed by external systems or clients.

5.3.6.8 Command to Run the Application

The `CMD` instruction specifies the command to start the Flask application:

flask run --host=0.0.0.0 --port=5000

5.3.7 AWS Auto scaling Group (ASG)

This setup is designed to deploy and manage a Flask-based application using Docker on AWS infrastructure, utilizing Auto Scaling Groups (ASG) for automatic scaling, Elastic Container Registry (ECR) for Docker image storage, and EC2 instances for hosting the application.

5.4. Summary

Chapter 5, Implementation, provides a comprehensive overview of the technical foundation underpinning the Easy Build mobile application. It meticulously details the selection and utilization of key tools, libraries, frameworks, and APIs that were instrumental in its development. Notably, the application leverages the powerful LangChain framework, chosen for its ability to streamline the creation of Large Language Model (LLM)-driven applications.

CHAPTER 6

TESTING, ANALYSIS AND VALIDATION

6.1 Introduction

This chapter provides an overview of the testing techniques used to evaluate the system's performance and functionality. Both Black-box and White-box testing methods are employed to ensure comprehensive coverage. The primary functionalities of the system are rigorously tested by creating well-structured test data and cases. Each test case is thoroughly documented and explained with the help of detailed tables, ensuring clarity and traceability throughout the testing process.

6.2 Test Methodology

This section describes the testing methodology used to evaluate the functionality and performance of the Chrome plugin. Software testing ensures that all plugin features, such as processing data from websites, PDF files, and YouTube videos, operate correctly by assessing the plugin's behavior and response to various user inputs. The accuracy of the plugin's responses and its ability to fetch and process data efficiently are analyzed in real-world settings.

6.3 Test Bed

Testing a specific software module, such as a function, class, or feature, in a controlled and defined manner is referred to as the "test bed." For this project, the test bed includes essential components such as a PC or laptop with a Chrome browser, necessary operating system configurations, and the plugin installed. The setup ensures comprehensive testing of the Chrome plugin's functionality, including its ability to process data from websites, PDF files, and YouTube videos.

6.4 Test Cases

Testing involves a series of systematic activities performed to verify whether the system meets its requirements and performs its functions correctly. Each test case is uniquely designed for a specific scenario and is executed independently for different modules of the system. Test cases are defined by several attributes, including a test case ID, input data, the partition being tested, actual output, expected output, and the outcome. The software tester

evaluates each test case to determine whether it passes or fails, ensuring that the plugin's functionalities such as processing data from websites, PDF files, and YouTube videos operate as intended and meet user expectations.

6.4.1 Plugin Usage on Any Website

Table 6.1 outlines the specific test cases designed to evaluate the plugin's functionality on any website. The tests ensure that the plugin can fetch website data and respond accurately to user queries based on the available content.

Table 6.1: Test case Usage on Any Website

Test Case ID	TC-1	
Use Case Reference	UC-1	
Test Date	05-12-2024	
Revision History	None	
Objective	To verify that the plugin works on any website page.	
Environment	User's Chrome browser	
Assumptions	The plugin has been installed successfully in the user's browser.	
Pre-Requisite	• The plugin is installed in the user's Chrome browser. • The user is on a website page.	
Steps	**Execution Description**	**Procedure Result**
	• The user navigates to any website page. • The user activates the plugin from the browser. • The user asks a query related to the content of the page.	• The plugin fetches the data from the current page and give to back-end services . • If the data contains the answer, the plugin provides it to the user.

Status	**Pass √**	**| Fail |**	**Not Executed**

6.4.2 Plugin Usage on Any PDF File

Table 6.2 details the test cases designed to verify the plugin's ability to process and fetch data from PDF files. The tests confirm that the plugin can accurately extract content from PDFs and provide relevant answers to user queries.

Table 6.2: Test Case chat with pdf file

Test Case ID	TC-2	
Use Case Reference	UC-2	
Test Date	05-12-2024	
Revision History	None	
Objective	To verify that the plugin works on any PDF file.	
Environment	User's Chrome browser	
Assumptions	The plugin has been installed successfully in the user's browser.	
Pre-Requisite	• The plugin is installed in the user's Chrome browser. 4. The user has opened a PDF file in the browser.	
Steps	**Execution Description**	**Procedure Result**
	• The user opens a PDF file in the Chrome browser. • The user activates the plugin from the browser. • The user asks a query related to the content of the PDF.	• The plugin fetches the data from the current PDF file and give to back-end services. • If the data contains the answer, the plugin provides it to the user. available.
Status	**Pass √ | Fail | Not Executed**	

6.4.3 Plugin Usage on Any YouTube Video

Table 6.3 describes the test cases aimed at assessing the plugin's functionality on YouTube videos. The tests validate that the plugin can fetch video transcripts and process user queries based on the extracted content.

Table 6.3: Test case Usage on Any YouTube Video

Test Case ID	TC-3	
Use Case Reference	UC-3	
Test Date	05-12-2024	
Revision History	None	
Objective	To verify that the plugin works on any YouTube video.	
Environment	User's Chrome browser	
Assumptions	The plugin has been installed successfully in the user's browser.	
Pre-Requisite		
Steps	**Execution Description**	**Procedure Result**
	• The plugin is installed in the user's Chrome browser. • The user has opened a YouTube video.	• The user opens a YouTube video in the Chrome browser and give to back-end services. • The user activates the plugin from the browser.
Status	**Pass √ | Fail | Not Executed**	

6.5 Summary

In Chapter 6, the Chrome plugin allows users to fetch data from any currently open website, PDF file, or YouTube video and provides answers to their queries based on the fetched content. If the requested information is not available, the plugin informs the user.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Introduction

This chapter provides a summary of the project, highlighting how the chatbot interacts with various applications and showcasing the system's functionality. It also discusses future plans to enhance this Chrome extension. Our Future goal is to deploy all its features to a server, ensuring wider accessibility. We aim to launch it as soon as possible, making it available to all Chrome extension users. This will save users time and boost productivity by leveraging the app’s capabilities.

7.2 System Overview

Web-Pilot emerges as a sophisticated Chrome extension, revolutionizing the online experience by seamlessly integrating an intelligent chatbot directly into the user's browser. This extension transcends traditional browsing by transforming passive content consumption into an active, conversational interaction. At its core, Web-Pilot leverages cutting-edge machine learning algorithms, enabling the chatbot to continuously refine its understanding and response accuracy with each user interaction. This adaptive learning mechanism ensures that the chatbot's assistance becomes progressively more insightful and tailored to individual needs. Web-Pilot's functionality extends beyond simple question-answering. It excels at extracting and synthesizing information from diverse online sources, including PDFs, entire websites, and even YouTube video transcripts. This capability empowers users to quickly grasp complex information without laborious reading or viewing. By simply posing questions in natural language, users can receive concise and relevant answers, effectively turning the vast expanse of the internet into a readily accessible knowledge base. Web-Pilot prioritizes user-friendliness, aiming to streamline everyday online tasks. Its intuitive interface and seamless integration into the Chrome browser make it an accessible tool for users of all technical backgrounds. Whether summarizing lengthy documents, clarifying complex concepts on a website, or extracting key points from a video, Web-Pilot acts as a reliable and efficient digital companion. By bridging the gap between passive browsing and interactive information retrieval, Web-Pilot significantly enhances productivity and empowers users to navigate the digital landscape with greater ease and understanding.

7.3 Milestone Achieved

The accomplished milestones in our chrome extension include:

7.3.1 Develop Browser extension

A functional Chrome extension was developed to provide users with a seamless interaction experience.

7.3.2 LLM Based Chatbot

An advanced chatbot powered by Large Language Models (LLMs) was integrated, enabling intelligent and accurate responses.

7.3.3 Dynamically Train on any text data

The chatbot can dynamically adapt and train on new text data for personalized and context-specific interactions.

7.3.3 Chat with any Large Contents

Web-Pilot allows users to interact with extensive content, such as large PDF documents and comprehensive web pages.

7.4 Limitations

While Web-Pilot offers numerous features, there are a few limitations:

1. Scanned PDFs or Non-Text PDFs The extension cannot process scanned PDFs or files without embedded text data.

2. Non-English Transcripts: It cannot fetch text from YouTube videos that lack transcripts or have transcripts in languages other than English.

3. Large Contexts: When dealing with very large content exceeding 28,000 words, the chatbot may take a longer time to process and could provide inaccurate answers.

7.5 Future Work

Our future plans for Web-Pilot include:

1. Server Deployment: Deploying all features to a centralized server for improved performance and accessibility.

2. Wider Launch: Making the extension available to all Chrome users, allowing them to benefit from its time-saving and productivity-enhancing features.

3. Improved Language Support: Enhancing support for multiple languages, including non-English transcripts, to broaden usability.

4. Optimized Large Context Processing: Improving the system's ability to handle large texts efficiently and accurately.

7.6 Summary

This chapter summarized the project's achievements, system capabilities, and current limitations while outlining future goals to improve the Web-Pilot extension. These advancements aim to create a more accessible, efficient, and user-friendly tool for navigating and interacting with online content.

APPENDICES

Appendix - I: Glossary

AI (Artificial Intelligence): Simulation of human intelligence processes by machines, especially computer systems.

Chatbot: A software application used to conduct an online chat conversation via text or text-to-speech, integrated with Web-Pilot for content interaction.

Chrome Extension: A small software program that customizes the browsing experience, such as the Web-Pilot.

Content Extraction: The process of retrieving data from various sources like web pages, YouTube videos, and PDF documents.

Generative AI: A type of AI technology that can generate text, images, or other content in response to prompts.

LLM (Large Language Model): Advanced AI models capable of understanding and generating human-like text, used in Web-Pilot for conversational interfaces.

PDF (Portable Document Format): A file format used to present and exchange documents reliably, often used for extracting data in Web-Pilot.

RAG (Retrieval-Augmented Generation): An AI training technique combining content retrieval and generative models to provide context-specific responses.

Scanned PDF: A PDF file containing images of text, making it non-readable for text extraction unless OCR (Optical Character Recognition) is applied.

Text Embeddings: Representations of text data in a high-dimensional space for semantic understanding and efficient processing.

Transcript: A textual representation of spoken content, such as subtitles from YouTube videos.

Vector Database: A specialized database for storing high-dimensional data representations like embeddings for efficient search and retrieval.

YouTube Video Transcripts: Text versions of the spoken content in YouTube videos, used by Web-Pilot for generating responses.

Workflow Diagram: A visual representation outlining processes, such as data scraping or chatbot integration.

User Query: A question or command entered by a user to retrieve information or perform actions via Web-Pilot.

Web Scraping: Automated extraction of content and data from web pages.

Dynamic Learning: An AI feature allowing models to adapt and refine understanding through continuous user interaction.

Fault Tolerance: The ability of a system to continue operating properly in the event of failures or errors.

Iterative Waterfall Methodology: A structured software development approach combining the sequential waterfall model with iterative cycles for refinement.

LangChain: A framework for building applications with LLMs, including features like prompt management and chaining multiple models together.

REFERENCES

[1] K. Lee and J. Yu, "A Systematic Review and Comprehensive Analysis of Pioneering AI Chatbot Models: ChatGPT, Bard, Llama," in *MDPI*, vol. 16, no. 7, pp. 219, 2024, doi: 10.3390/mdpi16(7)219.

[2] R. K. Smith, "A Systematic Review of Recent Research on Chatbots," in *IEEE Xplore*, 2024, doi: 10.1109/IEEEX.2024.10442532.

[3] J. White and K. McGarry, "A Literature Survey of Recent Advances in Chatbots," in *MDPI*, vol. 13, no. 1, pp. 41, 2024, doi: 10.3390/mdpi13(1)41.

[4] N. Zhang et al., "Emerging Trends in Chatbot Development: A Recent Survey," in *IEEE Xplore*, 2024, doi: 10.1109/IEEEX.2024.10307280.

[5] M. Davis et al., "ChatPapers: An AI Chatbot for Interacting with Academic Research," in *IEEE Xplore*, 2024, doi: 10.1109/IEEEX.2024.10470521.

[6] A. Patel et al., "The Role of Large Language Models in Enhancing Chatbot Efficiency," in *Future Internet*, 2024, doi: 10.3390/futureinternet2024.

[7] T. Howard, "Optimizing Natural Language Processing in AI Chatbots," in *Journal of AI Research*, 2024, [Online]. Available: https://www.journalofai.org.

[8] L. Grayson et al., "Innovations in Retrieval-Augmented Generation for AI Chatbots," in *AI Systems Quarterly*, 2024, [Online]. Available: https://aisystemsquarterly.com.

[9] P. Roberts, "Exploring Chatbot Integration with Web Browsers," in *Journal of Computational Integration*, 2024, [Online]. Available: https://compintegration.org.

[10] K. Thomas, "YouTube Video Transcription and Chatbot Applications," in *Multimedia AI Research*, 2024, [Online]. Available: https://multimediaai.org.